Cloud - AWS

Introduction to cloud computing

What is cloud computing

- Cloud computing is essentially the on-demand delivery of IT resources over the internet with pay-as-you-go pricing.
- Let's say you have an application that you have to run, content you need stored, or data you need analyzed.
- All of that data is stored in a data center, which is essentially a building or set of buildings devoted to housing servers that contain all this data.
- Data centers are designed with redundant power, cooling, and security measures to ensure secure and continuous operation.
- Historically, businesses would run applications in their own data centers or co-locate with other customers in a shared facility.
- Once AWS became available, companies could now run their applications in other data centers they didn't actually own.

Public, private, hybrid cloud models

- A public cloud is owned and operated by a third-party provider that delivers computing resources over the internet. These resources are shared in a multitenant environment, meaning multiple organizations use the same physical infrastructure. However, each customer's data and workloads are logically isolated to ensure privacy and security.
- **Advantages of public clouds:**
- No upfront hardware investment: Public cloud services follow a pay-as-you-go model, allowing businesses to avoid capital expenditures and start quickly.
- Global scalability: Providers like Microsoft Azure offer datacenters worldwide, supporting rapid scaling and geographic flexibility.
- High availability and reliability: Built-in redundancy, automated failover, and disaster recovery capabilities help maintain uptime and business continuity.
- Fully managed infrastructure: The cloud service provider handles hardware maintenance, software updates, patching, and capacity planning.

Public, private, hybrid cloud models

- A private cloud is a dedicated cloud environment used exclusively by a single organization. It can be hosted on-premises in a company's own datacenter or off-site by a third-party provider. Unlike public clouds, private clouds operate in a single-tenant environment, meaning all computing resources—servers, storage, and networking—are isolated and not shared with other organizations. This setup provides greater control, customization, and security.

Advantages of private clouds:

- Maximum control: Organizations have full authority over infrastructure, configurations, and security policies, allowing for tailored environments that meet specific business needs.
- Predictable performance
- Customizable infrastructure

Common use cases:

- Running sensitive workloads with strict compliance needs.
- Hosting regulated data, such as patient health records or financial transactions.
- Organizations often choose private cloud for its security, control, and compliance advantages, while using public cloud for less sensitive or more elastic workloads.

Hybrid clouds

- A hybrid cloud combines elements of public and private clouds, allowing data and applications to move between them seamlessly. This flexible architecture reflects the strengths of the public cloud vs. private cloud vs. hybrid cloud models—supporting sensitive workloads in a private cloud while using the public cloud for scalability, testing, or backup.

Common use cases:

- Seasonal businesses with fluctuating demand.
- Enterprises adopting a phased approach to gaining the benefits of cloud migration.
- Organizations needing local data residency but global application reach.
- Development and testing environments that benefit from public cloud agility.
- Disaster recovery strategies that replicate workloads across environments for business continuity.

History of public cloud – AWS version

- Early 2000s: Amazon.com operated primarily as an e-commerce website selling books and consumer goods
- Growing demand: Increased user traffic required constant IT upgrades (servers, storage, compute)
- Internal innovation: Amazon's IT team created standardized tools and processes to improve scalability and efficiency
- 2003 insight: Employees realized these internal solutions could benefit other companies

History of public cloud – AWS version

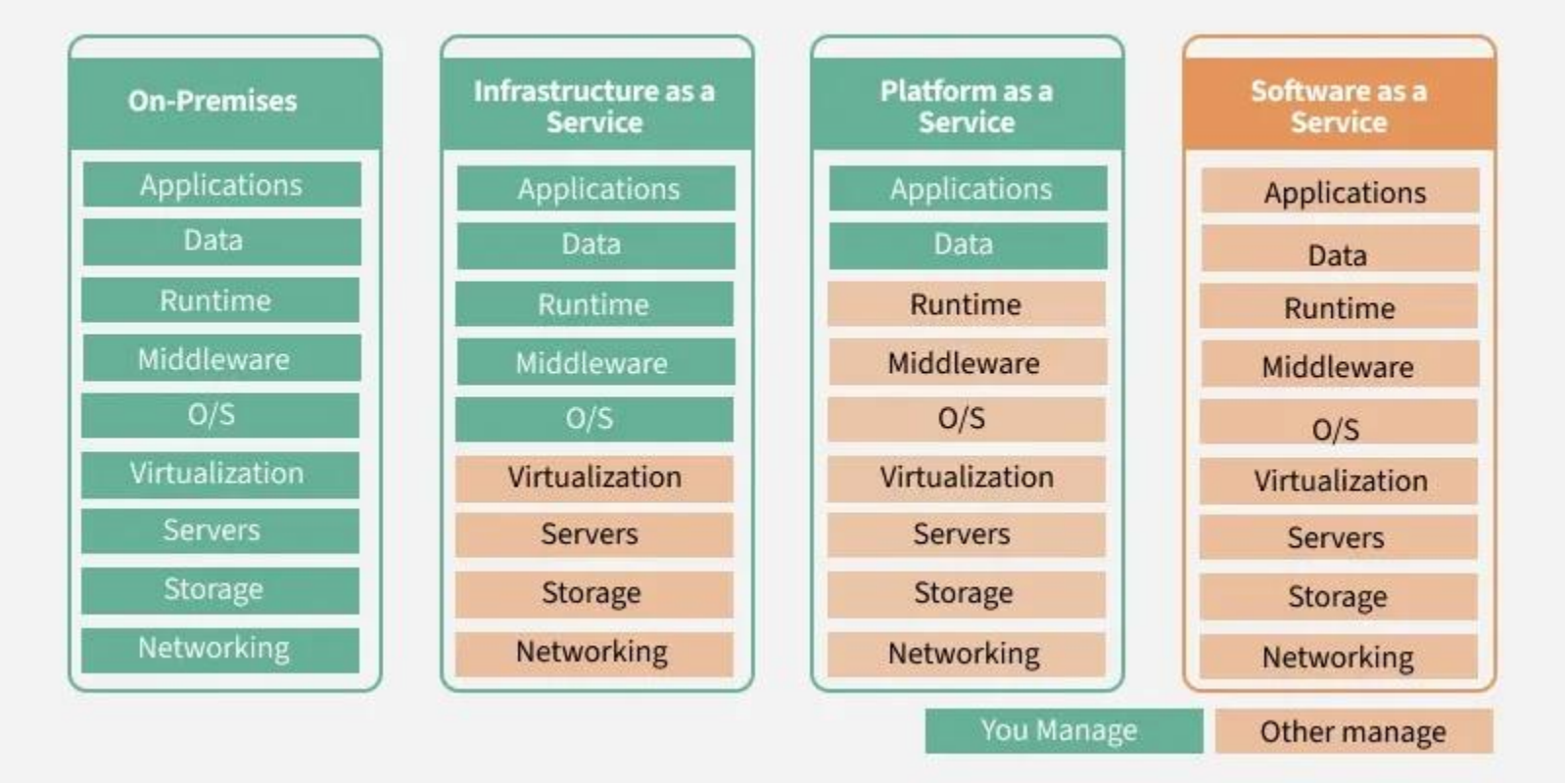
- New vision: Idea emerged to offer on-demand computing resources without upfront hardware investment
- 2004 milestone: AWS launched its first service, Amazon Simple Queue Service (SQS)
- 2006 expansion: Introduction of Amazon S3 (storage) and Amazon EC2 (compute)Early adopters: Initially popular with start-ups and developers
- Enterprise adoption: Larger organizations were drawn by scalability, cost savings, and ease of use
- Rapid growth: AWS expanded into databases, networking, analytics, and many other services

What is cloud computing

- Cloud computing is essentially the on-demand delivery of IT resources over the internet with pay-as-you-go pricing.
- Let's say you have an application that you have to run, content you need stored, or data you need analyzed.
- All of that data is stored in a data center, which is essentially a building or set of buildings devoted to housing servers that contain all this data.
- Data centers are designed with redundant power, cooling, and security measures to ensure secure and continuous operation.
- Historically, businesses would run applications in their own data centers or co-locate with other customers in a shared facility.
- Once AWS became available, companies could now run their applications in other data centers they didn't actually own.

SaaS, PaaS and IaaS

Cloud computing provides on-demand IT resources that can easily scale. Its three service models - IaaS, PaaS, and SaaS - cover different needs, from managing infrastructure to using ready-made software.



SaaS

Software as a Service (SaaS) is the most user-friendly model, providing complete software applications hosted in the cloud. Instead of purchasing and installing software on individual devices, users can access applications over the internet. SaaS eliminates the need for businesses to install, maintain, or manage software themselves.

Real-World Use Cases

Companies use Salesforce for customer relationship management (CRM), Microsoft 365 for office productivity tools, and Zoom for communication and meetings.

Characteristics of SaaS

- Applications are ready to use, and updates and maintenance are handled by the provider.
- You access the software through a web browser or app, usually paying a subscription fee.
- It's convenient and requires minimal technical expertise, ideal for non-technical users.

PaaS

Platform as a Service (PaaS) is a cloud service model where the cloud provider delivers a complete application platform, and the developer focuses only on writing and deploying code.

You do not manage servers, operating systems, or runtime environments.

Bring your code, we handle everything else.

How PaaS Works (Conceptually)

- Developer writes code
- Pushes code (Git, CLI, UI)
- PaaS platform: Builds the application, Allocates compute resources, Configures runtime, Deploys and scales automatically. **Often uses containers internally (hidden from the user)**

Examples: Google App Engine, Heroku, etc.

IaaS

Infrastructure as a Service (IaaS) provides virtualized computing resources-such as servers, storage, and networking-over the internet on a pay-as-you-go basis.

Characteristics of IaaS

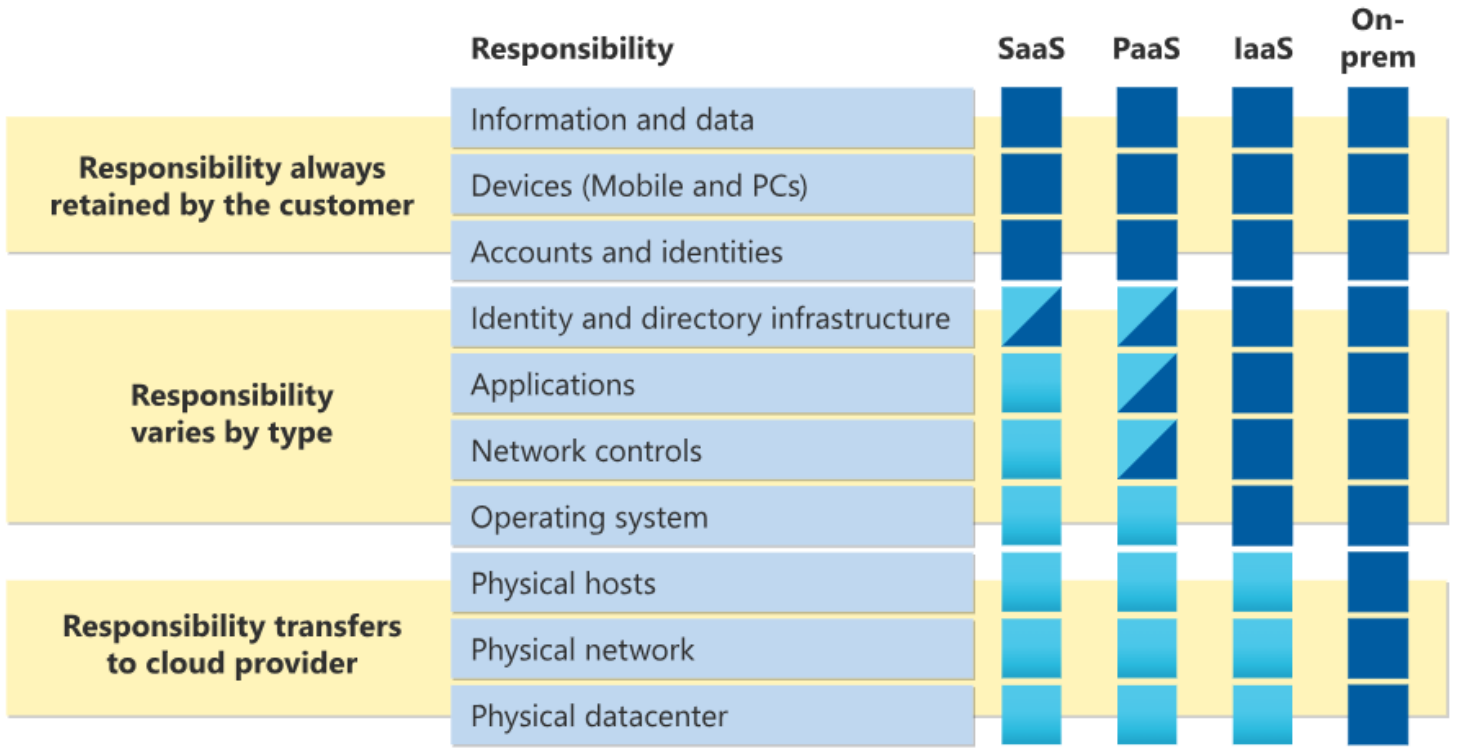
- IaaS is like renting virtual computers and storage space in the cloud.
- You have control over the operating systems, applications, and development frameworks.
- Scaling resources up or down is easy based on your needs.

Popular IaaS Providers

- Amazon Web Services
- Microsoft Azure
- Google Compute Engine
- Digital Ocean

Shared responsibility in the cloud

In an on-premises datacenter, you own the whole stack. As you move to the cloud some responsibilities transfer to cloud. The following diagram illustrates the areas of responsibility between you and the cloud, according to the type of deployment of your stack.



How to provision AWS resources

- In AWS, everything is an API call. An API is an application programming interface that defines predetermined ways for you to interact with AWS services.
- There are three main ways you could call AWS APIs: using the AWS Management Console, the AWS Command Line Interface, or CLI, or the AWS Software Development Kit, or SDK.
- Using the console, you can manage your resources visually and in a way that is easy to digest.
- Next up, you can use the AWS CLI to invoke AWS APIs using a terminal. This makes automation through scripting possible.

Regions, Availability Zones, and Local Zones

- Amazon cloud computing resources are hosted in multiple locations world-wide. These locations are composed of AWS Regions, Availability Zones, and Local Zones. Each AWS Region is a separate geographic area. Each AWS Region has multiple, isolated locations known as Availability Zones.

Regions

- Each Region is designed to be isolated from the other Regions. This achieves the greatest possible fault tolerance and stability.
- Most AWS services support regional resources. A regional resource is specific to the Region in which you create it. When you view your AWS resources, you must specify a Region, and then you see only the resources that are tied to that Region. You can replicate some types of resources across Regions, but we don't automatically replicate them for you.

Regions, Availability Zones, and Local Zones

Availability Zones

- Each Region has multiple, independent locations known as Availability Zones. The Availability Zones in a Region are connected through low-latency, high-bandwidth, highly-redundant networking, over dedicated metro fiber.
- Each Availability Zone consists of one or more discrete data centers, each with redundant power, networking, and connectivity, and housed in separate facilities. Because they are physically separate, only a single Availability Zone would be affected in the unlikely event of a fire, tornado, or flooding.
- Each Region has at least three Availability Zones. This helps you to design highly available applications on AWS.

Core AWS Service Categories

- AWS offers 200+ services, which can overwhelm beginners.To understand AWS, services should be grouped by function.

At a high level, AWS services are grouped into:

- Compute – Where code runs
- Storage – Where data lives
- Networking – How things connect
- Databases – How data is structured and queried
- Security & Identity – Who can do what
- Monitoring & Management – How we observe and control systems

Compute Services (Running Applications)

Amazon EC2 (Elastic Compute Cloud)Concept

- EC2 is a virtual machine in the cloud.
- You choose: CPU type, Memory size, Operating system, Storage, Network placement

Auto Scaling

- Problem it solves, Traffic is not constant. Without auto scaling:
- Too few servers → downtime
- Too many servers → wasted money
- Solution - Auto Scaling:
- Automatically adds EC2 instances when load increases
- Automatically removes EC2 instances when load decreases

Compute Services (Running Applications)

AWS Lambda (Serverless Compute)

- Concept: You don’t manage servers at all. 1/ Upload code, 2/ Define when it runs, 3/ Pay only when it executes
- AWS: Manages servers, Handles scaling automatically
- When to use: Event-driven systems, APIs, Automation, Background jobs
- EC2 You manage OS Always running Manual scaling
- Lambda: No OS access Runs only on events Automatic scaling

Storage Services (Saving Data)

Different applications need different storage types:

- Files
- Disks
- Objects
- Shared storage

AWS separates storage by access pattern, not by size

Amazon S3 (Simple Storage Service): S3 is object storage.

- Stores: Files, Images, Videos, Backups, Logs
- Each object has: A unique key, Metadata, No fixed size limit (up to 5 TB per object)
- Key properties: Extremely durable, Highly scalable, Accessible via HTTP
- Typical uses: Static website hosting, Backup and archiving, Media storage, Data lakes

Storage Services (Saving Data)

EBS is a virtual hard disk for EC2.

- Attached to one EC2 instance
- Used as: OS disk, Database disk, Application disk
- EBS Block storage Attached to EC2 Low latency
- S3 Object storage Independent Higher latency

Elastic File System

- EFS is shared file storage.
- Multiple EC2 instances can access it
- Looks like a network drive
- Typical uses: Shared application files, Microservices, Content management systems

Networking Services (Connecting Everything)

Amazon VPC (Virtual Private Cloud)

- A VPC is your private network inside AWS.
- You control: IP address range, Subnets, Routing, Security
- VPC = your own data center network in the cloud

Subnets

- A subnet is a portion of a VPC
- Usually tied to one Availability Zone

Types:

- Public subnet → has internet access
- Private subnet → no direct internet access

Networking Services (Connecting Everything)

Internet Gateway (IGW)

Allows communication between: Public subnets and The internet

Without IGW: No inbound or outbound internet traffic.

NAT Gateway

Allows private subnet instances to: Access the internet, Without being accessible from the internet

Used for: Software updates, External API calls

Security Groups vs NACLs

Security Groups: Instance-level, Stateful,

Network ACLs: Subnet-level, Stateless, Advanced filtering

Database Services (Storing Structured Data)

Amazon RDS (Relational Database Service)

Managed relational databases.

Supported engines: MySQL, PostgreSQL, MariaDB, OracleSQL Server

AWS handles: Backups, Patching, Replication

You focus on: Schema, Queries Data

Amazon DynamoDB (NoSQL)

Concept: Fully managed key-value and document database.

Key features: Massive scalability, Single-digit millisecond latency, Serverless

Typical uses: Session storage, User profiles, IoT data

AWS Pricing Model

Core Pricing Principles in AWS

Pay-as-You-Go: You are charged based on: Time, Usage, Consumption

Examples: EC2 → per second or per hour

S3 → per GB per month

Data transfer → per GB

You stop using → you stop paying

Reserved Instances (RI)

Commit to 1 or 3 years

Lower hourly price

When to use: Steady workloads, Databases, Always-on systems

AWS Pricing Model

Spot Instances

Use unused AWS capacity

Very large discounts

Risk: Instance can be stopped anytime

When to use: Batch jobs, Data processing, Fault-tolerant workloads

Example of a Web App on AWS

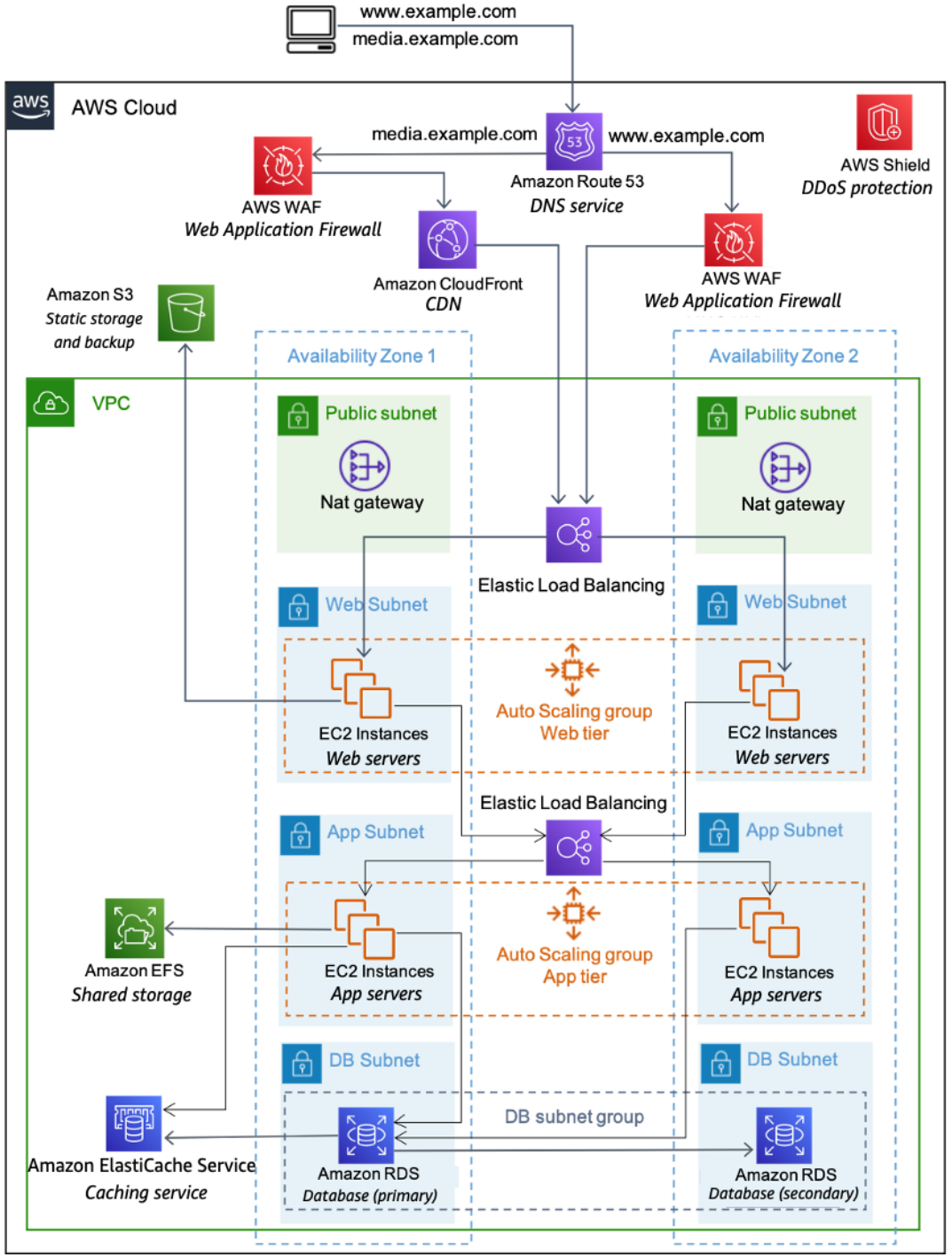
DNS services with Amazon Route 53 – Provides DNS services to simplify domain management.

Edge caching with Amazon CloudFront – Edge caches high-volume content to decrease the latency to customers.

Edge security for Amazon CloudFront with AWS WAF – Filters malicious traffic, including cross site scripting (XSS) and SQL injection via customer-defined rules.

Load balancing with ELB (ELB) – Enables you to spread load across multiple Availability Zones and Amazon EC2 Auto Scaling groups for redundancy and decoupling of services.

DDoS protection with AWS Shield – Safeguards your infrastructure against the most common network and transport layer DDoS attacks automatically.



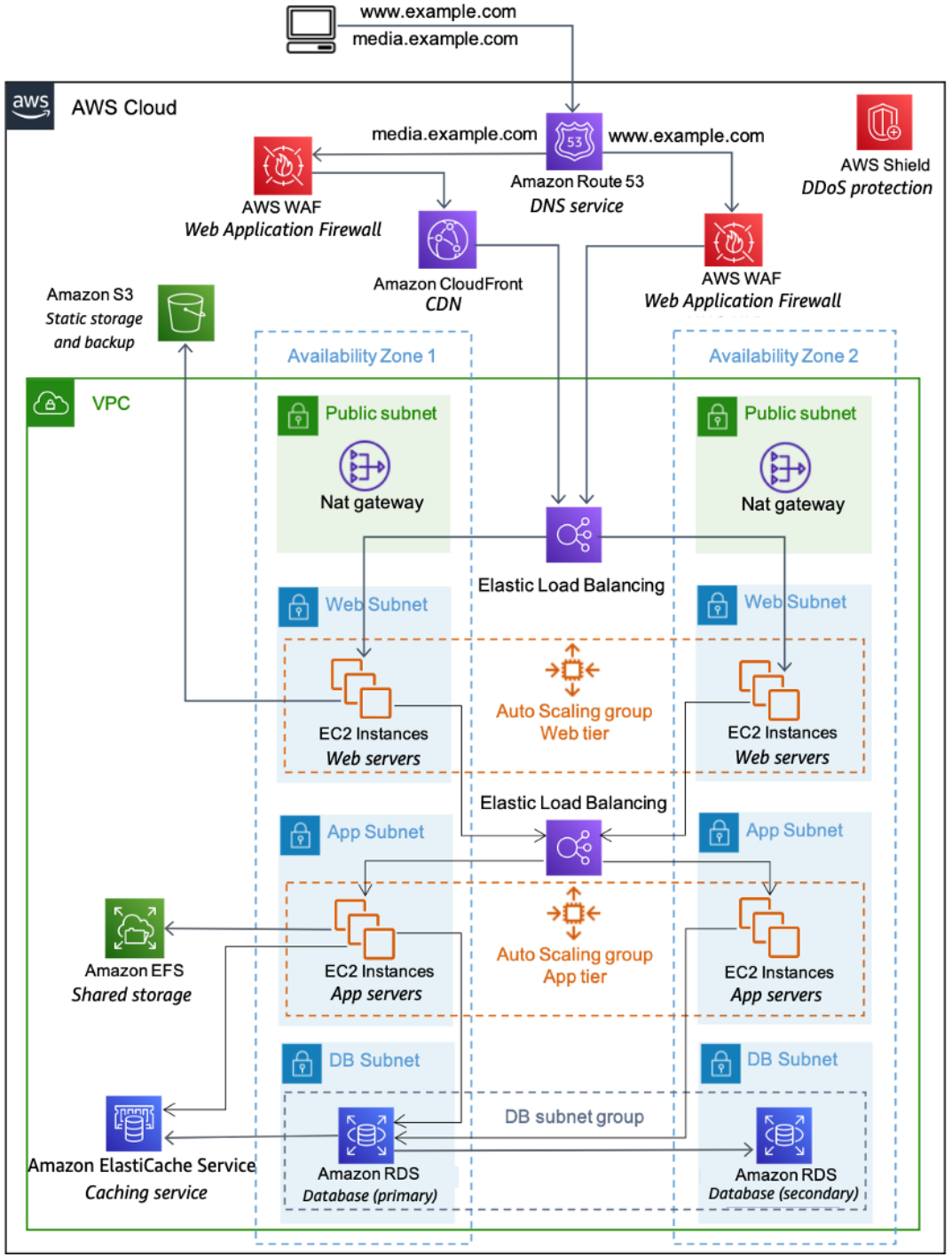
Example of a Web App on AWS

Firewalls with security groups – Moves security to the instance to provide a stateful, host-level firewall for both web and application servers.

Caching with Amazon ElastiCache – Provides caching services with Redis or Memcached to remove load from the app and database, and lower latency for frequent requests.

Managed database with Amazon Relational Database Service (Amazon RDS) – Creates a highly available, multi-AZ database architecture with six possible DB engines.

Static storage and backups with Amazon Simple Storage Service (Amazon S3) – Enables simple HTTP-based object storage for backups and static assets like images and video.



AWS Compute

EC2

- EC2 instances are virtual machines, or VMs. VMs share an underlying physical host machine with multiple other instances, which is a concept called multi-tenancy.
- In a multi-tenant environment, you need to make sure that each VM is isolated from each other but is still able to share resources provided by the host.
- This job of resource sharing and isolation is being done by a piece of software called a hypervisor, which is running on the host machine. For EC2, AWS manages the underlying host, the hypervisor, and the isolation from instance to instance.
- So, even though you won't be managing this piece, it's important to have a basic grasp of the concept of multi-tenancy.

EC2

- When you provision an EC2 instance, you can choose the operating system, or OS, based on either Windows or Linux.
- You also can configure what software you want running on the instance.
- EC2 instances are also resizable. You might start with a small instance and realize the application you are running is starting to max out that server. You can then give that instance more memory and more CPU. This is what we call **vertically scaling an instance**. In essence, you can make instances bigger or smaller whenever you need to.
- You also control the networking aspect of EC2. So, you decide what type of requests make it to your server and if they are publicly or privately accessible. We'll talk more about this later in the networking section.

EC2 instance types

- General purpose instances provide a good balance of compute, memory, and networking resources. They can be used for lots of diverse workloads, like web services or code repositories. They're also a good starting point if you don't know how your workload will perform ahead of time.
- Compute-optimized instances are ideal for compute-intensive tasks, like gaming servers, high-performance computing, machine learning tasks--even scientific modeling.
- Memory-optimized instances are good for memory-intensive tasks. They deliver fast performance for workloads that process large data sets in memory.

EC2 instance types

- Accelerated-computing instances are good for floating point number calculations, graphics processing, or data pattern matching. This is because they use hardware accelerators. These are co-processors that perform functions more efficiently than is possible in software running on CPUs.
- And, finally, storage-optimized instances are ideal for workloads that require high performance for locally stored data.

EC2 – Hypervisors and Orchestration

- A hypervisor is a software that you can use to run multiple virtual machines on a single physical machine. Every virtual machine has its own operating system and applications. The hypervisor allocates the underlying physical computing resources such as CPU and memory to individual virtual machines as required. Thus, it supports the optimal use of physical IT infrastructure.

Type 1 hypervisor

- The type 1 hypervisor sits on top of the metal server and has direct access to the hardware resources. Because of this, the type 1 hypervisor is also known as **a bare-metal hypervisor**. The host machine does not have an operating system installed in a bare-metal hypervisor setup. Instead, the hypervisor software acts as a lightweight operating system.

Type 2 hypervisor

The type 2 hypervisor is a hypervisor program installed on a host operating system. It is also known as a hosted or embedded hypervisor. Like other software applications, hosted hypervisors do not have complete control of the computer resources. Instead, the system administrator allocates the resources for the hosted hypervisor, which it distributes to the virtual machines.

EC2 – Hypervisors and Orchestration

- AWS primarily uses the Nitro Hypervisor, which is a lightweight Type 1 (bare-metal) hypervisor.

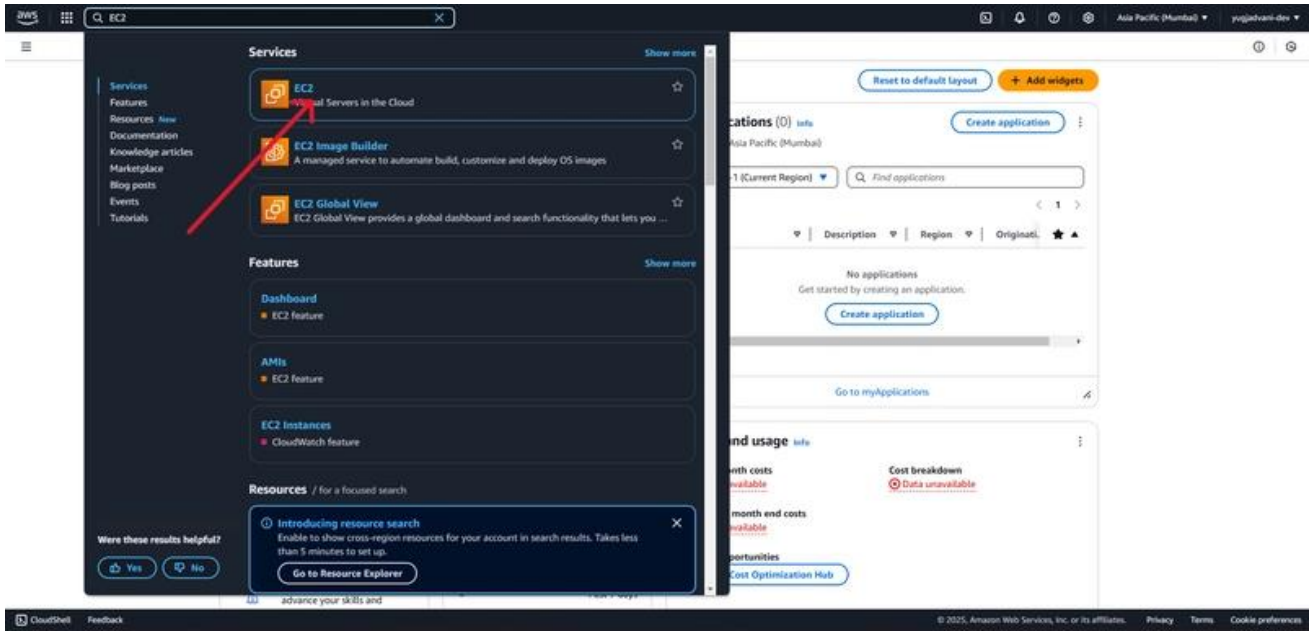
AWS Orchestrator

- In AWS, the orchestrator is part of the control plane. It doesn't virtualize hardware itself — that's the hypervisor's job — but it decides where and how your workloads run across AWS's massive infrastructure.
- Orchestrator is like a traffic manager
- The hypervisor handles resources on one server. The orchestrator decides which server your EC2 instance should run on, based on availability, load, and networking.

EC2 – Launch an instance example

1. Search for EC2 and Click on It

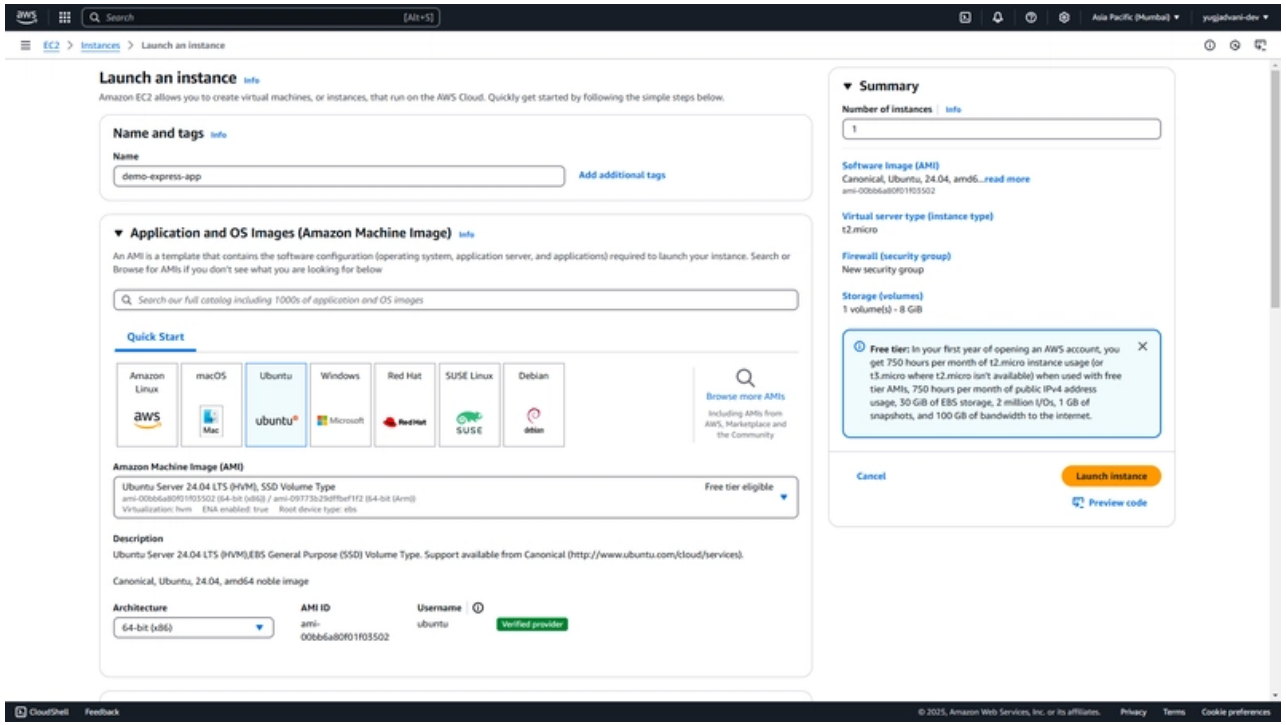
- Within the AWS Management Console, use the universal search bar to find "EC2.“
- Multi-Region Strategy: If you operate globally, ensure you're launching in the correct region to minimize latency and meet compliance needs.
- Once you land on the EC2 dashboard, select "Launch instance."



EC2 – Launch an instance example

2. Fill In the Details for Your Machine

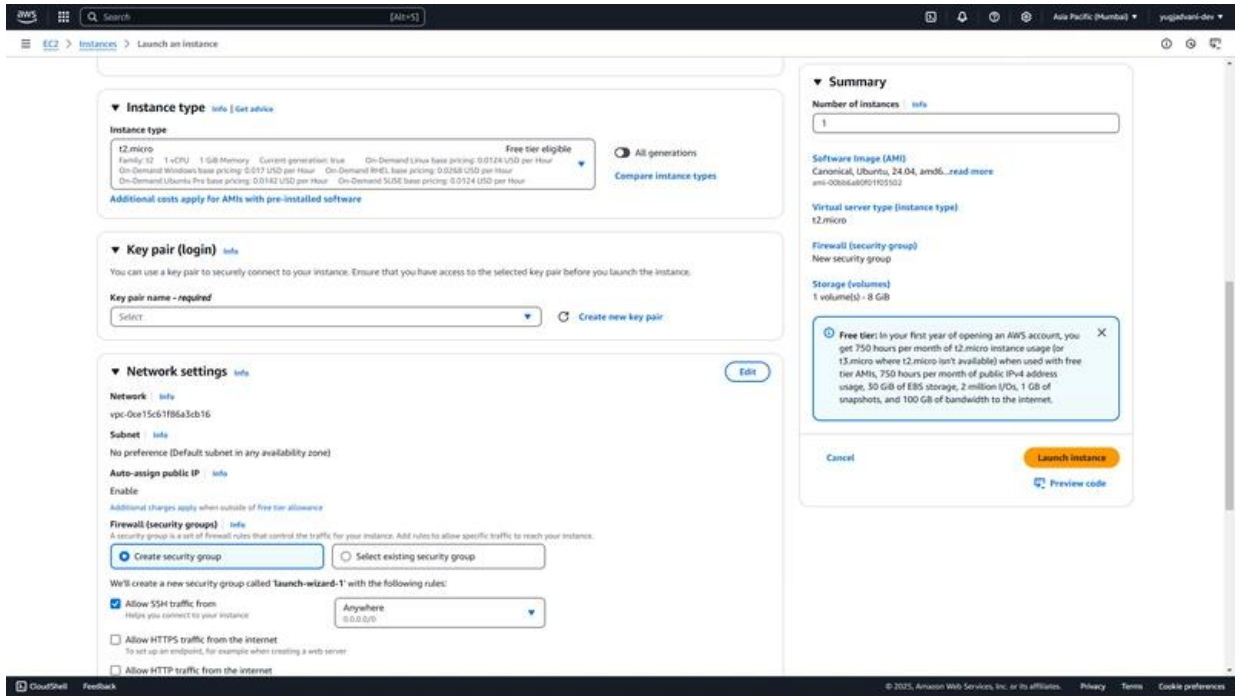
- Name your instance something meaningful, e.g., demo-express-app or production-analytics-node
- Choose the base operating system (e.g., Ubuntu, Amazon Linux, Windows Server).



EC2 – Launch an instance example

2. Instance Type

- EC2 offers a wide array of instance types (t2.micro, t3.medium, c5.xlarge, etc.) that vary by CPU, memory, storage, and network capacity.
- Free Tier: t2.micro (or t3.micro) is free-tier eligible for new accounts (750 hours/month).
- Workload Matching: Choose an instance type that matches your workload. For compute-intensive tasks, consider C-series. For memory-intensive tasks, consider R-series.
- Reserved vs. On-Demand: For predictable, long-running workloads, reserved instances or savings plans can significantly reduce costs.
- Elasticity: Start small and scale up as needed, especially for new or pilot projects.



EC2 – Launch an instance example

Key Pair (Login)

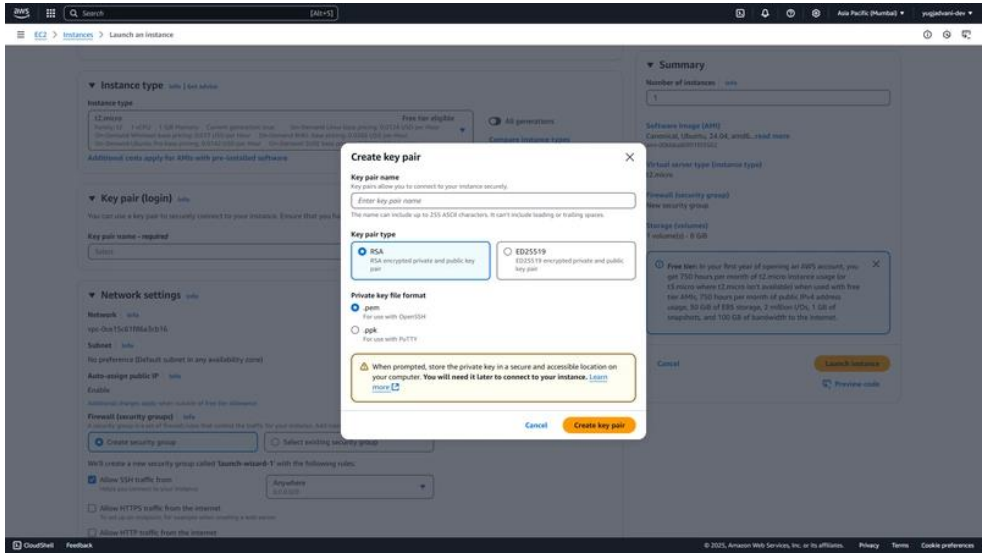
- You must have a key pair to securely SSH into your machine
- Create a New Key Pair: If you don't have one, create it and download the private key (.pem or .ppk).
- Security Best Practice: Store the private key in a secure location. If it's compromised, your instance could be at risk.

Security Groups

- act as a virtual firewall. By default, you can create a new security group that allows SSH (port 22) and your application port (e.g., port 3000).

Configure Storage

- By default, AWS provides 8 GiB of EBS (Elastic Block Store) storage in the free tier. You might increase this to 16 GiB or more, depending on your application's needs.



Launch first EC2 in default VPC

When you start using Amazon VPC, you have a default VPC in each AWS Region. A default VPC comes with a public subnet in each Availability Zone, an internet gateway, and settings to enable DNS resolution. Therefore, you can immediately start launching Amazon EC2 instances into a default VPC.

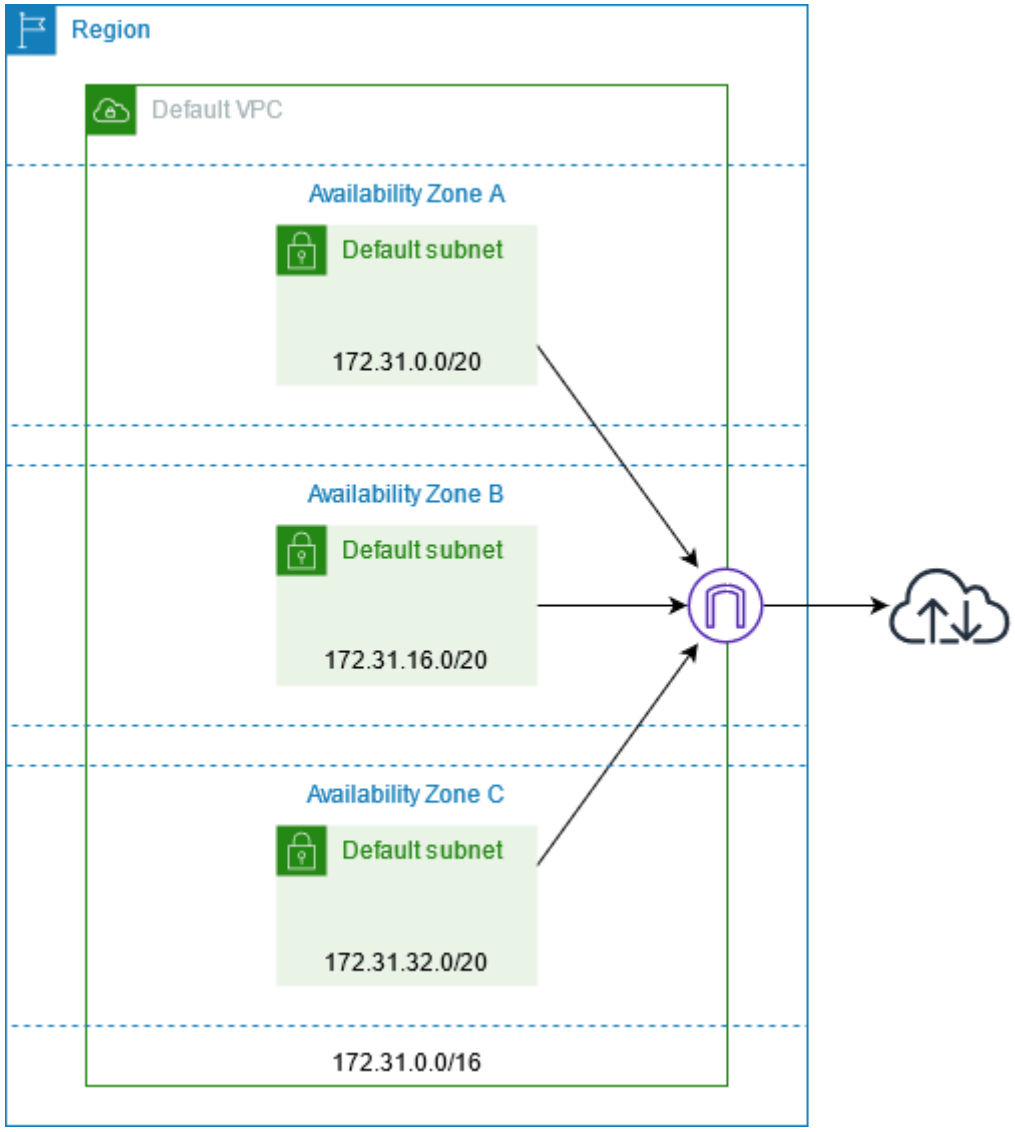
When we create a default VPC, we do the following to set it up for you:

- Create a VPC with a size /16 IPv4 CIDR block (172.31.0.0/16). This provides up to 65,536 private IPv4 addresses.
- Create a size /20 default subnet in each Availability Zone. This provides up to 4,096 addresses per subnet, a few of which are reserved for our use.
- Create an internet gateway and connect it to your default VPC.
- Add a route to the main route table that points all traffic (0.0.0.0/0) to the internet gateway.
- Create a default security group and associate it with your default VPC.
- Create a default network access control list (ACL) and associate it with your default VPC.
- Associate the default DHCP options set for your AWS account with your default VPC.

Launch first EC2 in default VPC

The following table shows the routes in the main route table for the default VPC.

Destination	Target
172.31.0.0/16	local
0.0.0.0/0	internet_gateway_id

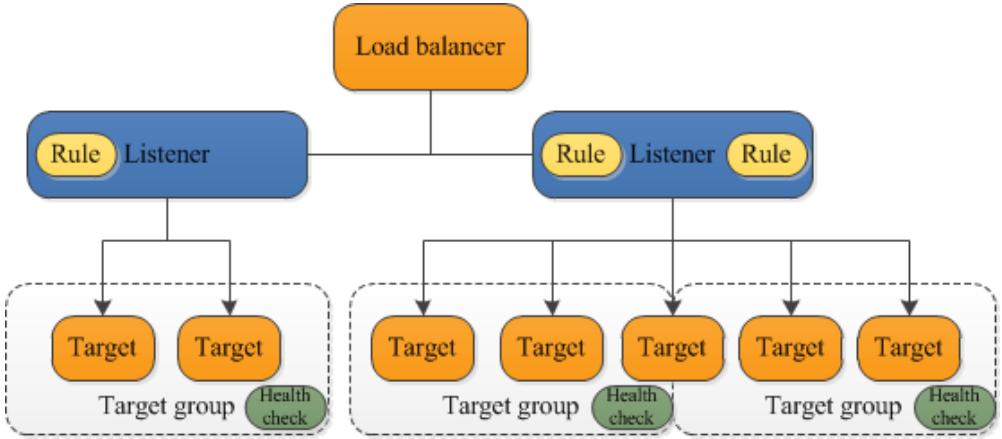


ELB – Elastic Load Balancer

- A load balancer accepts incoming traffic from clients and routes requests to its registered targets (such as EC2 instances) in one or more Availability Zones.
- The load balancer also monitors the health of its registered targets and ensures that it routes traffic only to healthy targets.
- When the load balancer detects an unhealthy target, it stops routing traffic to that target. It then resumes routing traffic to that target when it detects that the target is healthy again.
- ELB supports the following load balancers: Application Load Balancers, Network Load Balancers.
- A listener checks for connection requests from clients, using the protocol and port that you configure.

ELB – Elastic Load Balancer

- The rules that you define for a listener determine how the load balancer routes requests to its registered targets.
- Each rule consists of a priority, one or more actions, and one or more conditions.
- When the conditions for a rule are met, then its actions are performed. You must define a default rule for each listener, and you can optionally define additional rules.
- Each target group routes requests to one or more registered targets, such as EC2 instances, using the protocol and port number that you specify. You can register a target with multiple target groups.
- The following diagram illustrates the basic components. Notice that each listener contains a default rule, and one listener contains another rule that routes requests to a different target group. One target is registered with two target groups.



ELB – Application Load Balancer

- An Application Load Balancer (ALB) is a Layer 7 (Application layer) load balancer in AWS that distributes HTTP and HTTPS traffic to backend targets based on application-level rules.
- Unlike a traditional router or L4 load balancer, ALB understands HTTP: URLs
  - Headers
  - Hostnames
  - Methods (GET, POST, etc.)

Example:

Listener 1: HTTP : 80

Listener 2: HTTPS : 443

Each listener contains rules - Rules define how traffic is routed.

A rule has: Conditions (if…), Actions (then…)

Conditions can inspect: Path (/api/*), Host header (api.example.com), Etc.

Actions: Forward to a target group, Redirect (HTTP → HTTPS), Return a fixed response (e.g., 403), etc.

A target group is a logical group of backends.

Supported target types: EC2 instances, IP addresses, Lambda functions

ELB – Application Load Balancer

- You want to host: A frontend web app A backend REST API. Both are in the same VPC.

VPC: 10.0.0.0/16

Public Subnets:

- 10.0.1.0/24 (AZ-A)
- 10.0.2.0/24 (AZ-B)

Private Subnets:

- 10.0.11.0/24 (AZ-A)
- 10.0.12.0/24 (AZ-B)

Application Load Balancer: Attached to public subnets, Internet-facing, DNS name: my-alb-123456.elb.amazonaws.com

Backend EC2 Instances Located in private subnets, No public Ips, Security group allows traffic only from ALB

ELB – Application Load Balancer

Listener: HTTPS (443)

Rule 1 – API trafficI

- IF path is /api/*THEN forward to target group: api-tg

Rule 2 – Web traffic

- IF path is /*THEN forward to target group: web-tg

Target Groups

- web-tg Targets: EC2 instances on port 80
- api-tg Targets: EC2 instances on port 8080

ELB – Application Load Balancer

Let’s trace this request: https://example.com/api/users

Step 1: DNS Resolution

example.com → ALB DNS name

Client gets a public IP of ALB

Step 2: Traffic Enters ALB

Client connects to ALB on port 443

TLS is terminated at the ALB (SSL offloading)

Step 3: Rule Evaluation

ALB inspects:Path = /api/users

Matches:/api/*

Step 4: Forward to Target Group

ALB selects a healthy backend from api-tg

Forwards request to:10.0.11.25:8080

ELB – Network Load Balancer

A Network Load Balancer (NLB) is a Layer 4 (Transport layer) load balancer in AWS that distributes traffic based on IP address and port only, without inspecting application data.

Unlike ALB, NLB:

- Does not understand HTTP
- Does not look at URLs, headers, or cookies
- Works with any TCP/UDP-based protocol

This makes NLB ideal for:

- Ultra-low latency systems
- High-throughput workloads
- Non-HTTP protocols
- Applications that need client source IP preservation

ELB – Availability Zones and load balancer nodes

When you enable an Availability Zone for your load balancer, ELB creates a load balancer node in the Availability Zone.

If you register targets in an Availability Zone but do not enable the Availability Zone, these registered targets do not receive traffic.

Your load balancer is most effective when you ensure that each enabled Availability Zone has at least one registered target.

We recommend enabling multiple Availability Zones for all load balancers. With an Application Load Balancer however, it is a requirement that you enable at least two or more Availability Zones.

This configuration helps ensure that the load balancer can continue to route traffic. If one Availability Zone becomes unavailable or has no healthy targets, the load balancer can route traffic to the healthy targets in another Availability Zone.

After you disable an Availability Zone, the targets in that Availability Zone remain registered with the load balancer. However, even though they remain registered, the load balancer does not route traffic to them.

ELB – Availability Zones and load balancer nodes

First Principle: ELB Is Regional, Not Zonal

- An AWS load balancer is: One logical resource
- Regional Implemented using multiple AZ-local nodes
- **You never create “one load balancer per AZ”, AWS creates load balancer nodes in each enabled AZ**
- Subnets = AZ Enablement
- When you create an ELB, you must select subnets.

Example:

- Subnet-1 → AZ-a
- Subnet-2 → AZ-b
- Subnet-3 → AZ-c

This is how you tell AWS which AZs to use.

ELB – Availability Zones and load balancer nodes

AWS does internally

For each selected subnet (AZ), AWS deploys managed LB nodes

Each node has:

- ENIs
- Private Ips
- Public IPs (if internet-facing)

You never see or manage these nodes.

DNS Is the Control Plane (Very Important), One DNS namemy-alb-123456.elb.amazonaws.com

DNS resolves to multiple Ips: AZ-a → IP-A, AZ-b → IP-B, AZ-c → IP-C

AWS Route 53 (internal) manages this. DNS is how AWS spreads traffic across AZs.

ELB – Availability Zones and load balancer nodes

Client Traffic Flow (End-to-End)

Step 1: DNS Resolution

Client resolves: my-alb.amazonaws.com

Receives multiple IPs, one per AZ.

Step 2: Client Chooses an IP

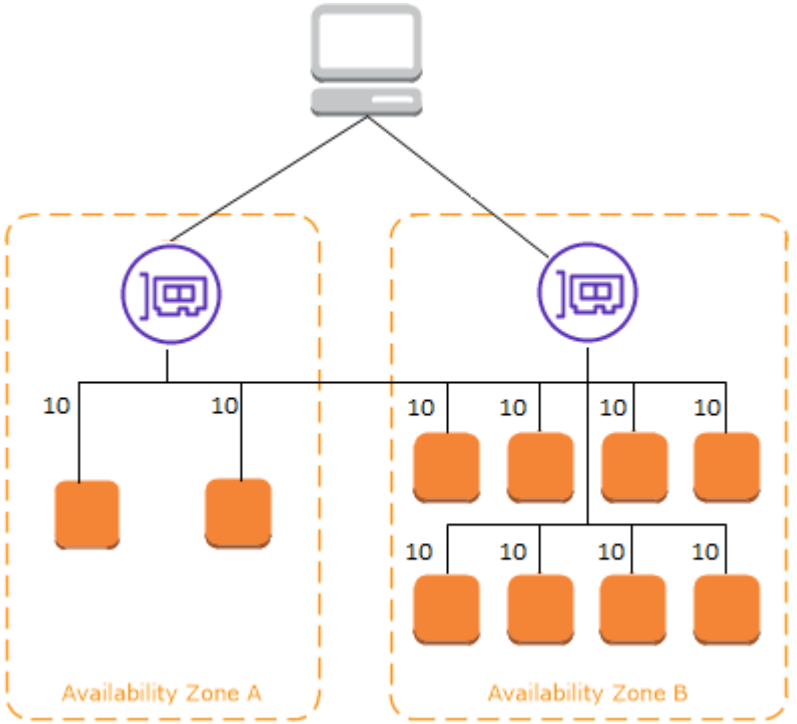
Client picks one IP (usually nearest / random)Connects directly to that AZ’s LB node

This is why ELB scales automatically.

ELB – Cross-zone load balancing

The nodes for your load balancer distribute requests from clients to registered targets. When cross-zone load balancing is enabled, each load balancer node distributes traffic across the registered targets in all enabled Availability Zones. When cross-zone load balancing is disabled, each load balancer node distributes traffic only across the registered targets in its Availability Zone.

The following diagrams demonstrate the effect of cross-zone load balancing with round robin as the default routing algorithm. There are two enabled Availability Zones, with two targets in Availability Zone A and eight targets in Availability Zone B. Clients send requests, and Amazon Route 53 responds to each request with the IP address of one of the load balancer nodes. Based on the round robin routing algorithm, traffic is distributed such that each load balancer node receives 50% of the traffic from the clients. Each load balancer node distributes its share of the traffic across the registered targets in its scope.



ELB – Cross-zone load balancing

If cross-zone load balancing is enabled, each of the 10 targets receives 10% of the traffic. This is because each load balancer node can route its 50% of the client traffic to all 10 targets.

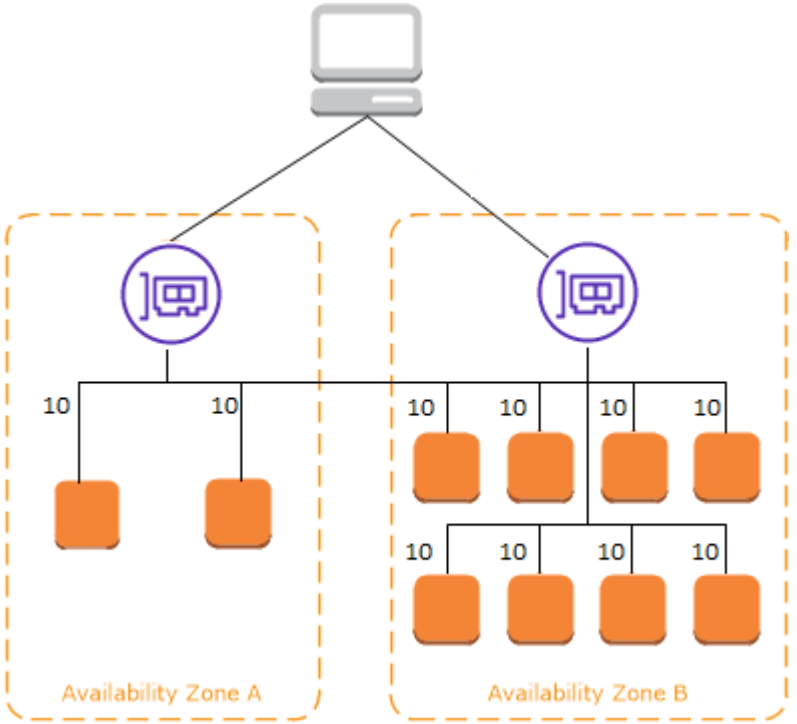
If cross-zone load balancing is disabled:

- Each of the two targets in Availability Zone A receives 25% of the traffic.
- Each of the eight targets in Availability Zone B receives 6.25% of the traffic.

This is because each load balancer node can route its 50% of the client traffic only to targets in its Availability Zone.

With Application Load Balancers, cross-zone load balancing is always enabled at the load balancer level.

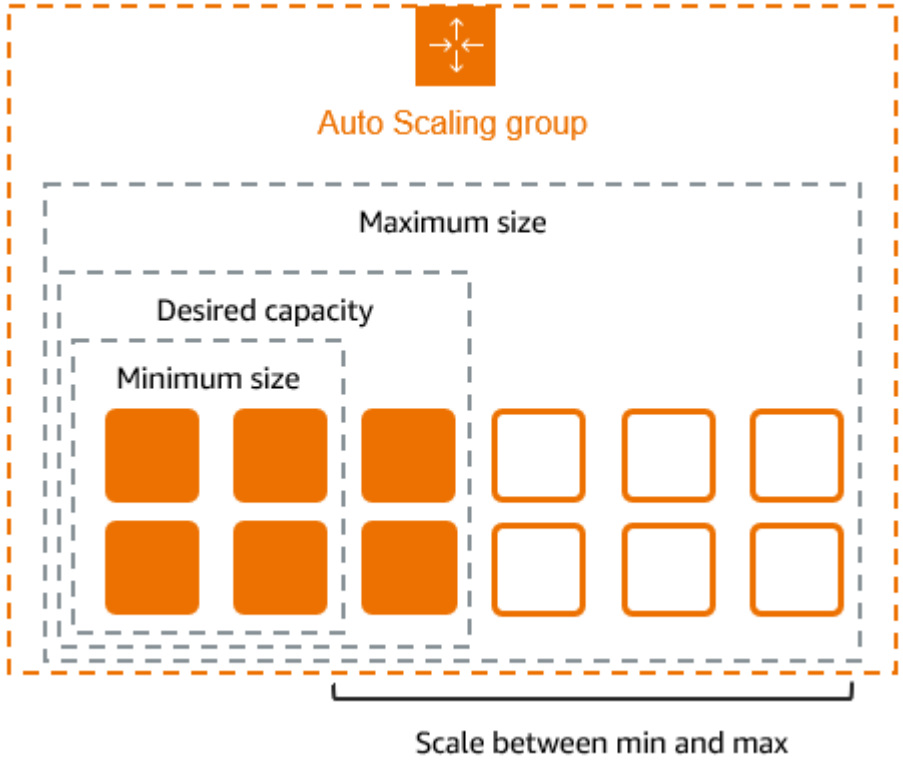
With Network Load Balancers and Gateway Load Balancers, cross-zone load balancing is disabled by default.



Amazon EC2 Auto Scaling

Amazon EC2 Auto Scaling helps you ensure that you have the correct number of Amazon EC2 instances available to handle the load for your application. You create collections of EC2 instances, called Auto Scaling groups. You can specify the minimum number of instances in each Auto Scaling group, and Amazon EC2 Auto Scaling ensures that your group never goes below this size. You can specify the maximum number of instances in each Auto Scaling group, and Amazon EC2 Auto Scaling ensures that your group never goes above this size.

For example, the following Auto Scaling group has a minimum size of four instances, a desired capacity of six instances, and a maximum size of twelve instances. The scaling policies that you define adjust the number of instances, within your minimum and maximum number of instances, based on the criteria that you specify.

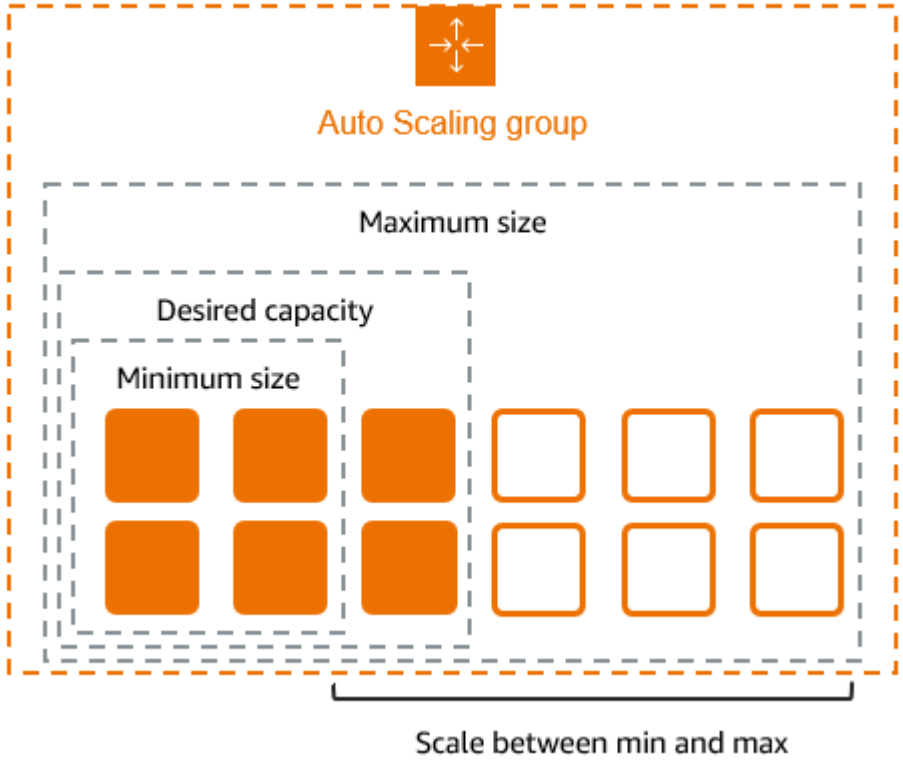


Amazon EC2 Auto Scaling

An Auto Scaling Group (ASG) is responsible for: Launching EC2 instances, Terminating EC2 instances, Keeping the desired number of instances, Scaling in and out based on policies

It does NOT distribute traffic.

You need to attach a load balancer and put the scaling group in the target group.



Introduction to Serverless

- One of the major benefits of cloud computing is its ability to abstract (hide) the infrastructure layer. This ability eliminates the need to manually manage the underlying physical hardware.
- In a serverless environment, this abstraction allows you to focus on the code for your applications without spending time building and maintaining the underlying infrastructure.
- With serverless applications, there are never instances, operating systems, or servers to manage.
- AWS handles everything required to run and scale your application.
- By building serverless applications, your developers can focus on the code that makes your business unique.

Deployment and Operational tasks	Traditional Environment	Serverless
Configure an instance	YES	-
Update operating system (OS)	YES	-
Install application platform	YES	-
Build and deploy apps	YES	YES
Configure automatic scaling and load balancing	YES	-
Continuously secure and monitor instances	YES	-
Monitor and maintain apps	YES	YES

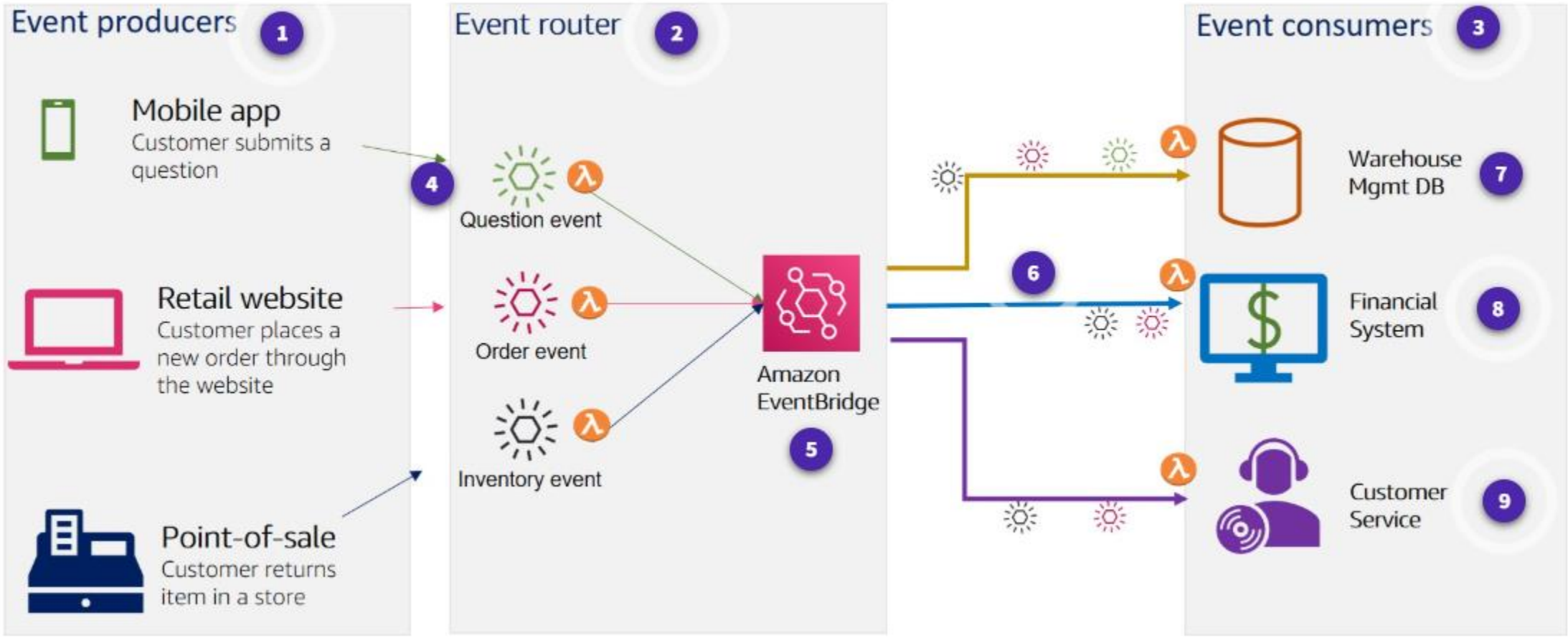
What is AWS Lambda?

- AWS Lambda is a compute service. You can use it to run code without provisioning or managing servers. Lambda runs your code on a high-availability compute infrastructure. It operates and maintains all of the compute resources, including server and operating system maintenance, capacity provisioning and automatic scaling, code monitoring, and logging. With Lambda, you can run code for almost any type of application or backend service.
- Some benefits of using Lambda include the following:
  - You can run code without provisioning or maintaining servers.
  - It initiates functions for you in response to events.
  - It scales automatically.
  - It provides built-in code monitoring and logging via Amazon CloudWatch.

Event-driven architectures

- An event-driven architecture uses events to initiate actions and communication between decoupled services. An event is a change in state, a user request, or an update, like an item being placed in a shopping cart in an e-commerce website.
- When an event occurs, the information is published for other services to consume it. In event-driven architectures, events are the primary mechanism for sharing information across services.
- AWS Lambda is an example of an event-driven architecture. Most AWS services generate events and act as an event source for Lambda.
- Lambda runs custom code (functions) in response to events. Lambda functions are designed to process these events and, once invoked, may initiate other actions or subsequent events.

Event-driven architectures



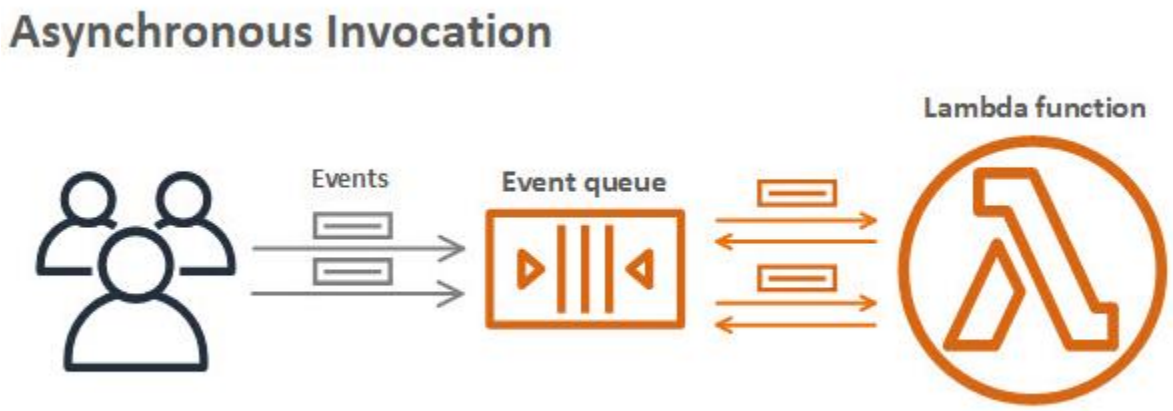
What is a Lambda function?

- The code you run on AWS Lambda is called a Lambda function. Think of a function as a small, self-contained application. After you create your Lambda function, it is ready to run as soon as it is initiated.
- Each function includes your code as well as some associated configuration information, including the function name and resource requirements.
- Lambda functions are stateless, with no affinity to the underlying infrastructure. Lambda can rapidly launch as many copies of the function as needed to scale to the rate of incoming events.
- After you upload your code to AWS Lambda, you can configure an event source, such as an Amazon Simple Storage Service (Amazon S3) event, Amazon DynamoDB stream, Amazon Kinesis stream, or Amazon Simple Notification Service (Amazon SNS) notification.
- When the resource changes and an event is initiated, Lambda will run your function and manage the compute resources as needed to keep up with incoming requests.

How AWS Lambda Works

Asynchronous events

- When you invoke a function asynchronously, events are queued and the requestor doesn't wait for the function to complete. This model is appropriate when the client doesn't need an immediate response.



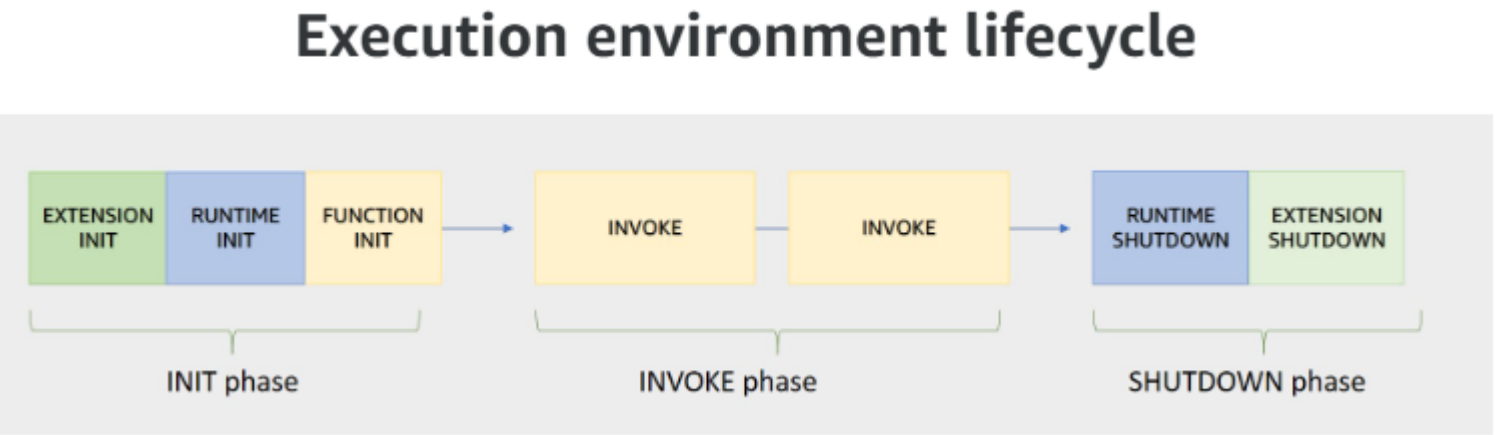
How AWS Lambda Works

Polling invocation

- This invocation model is designed to integrate with AWS streaming and queuing based services with no code or server management.
- Lambda will poll (or watch) these services, retrieve any matching events, and invoke your functions. This invocation model supports the following services:
  - Amazon Kinesis
  - Amazon SQS
  - Amazon DynamoDB Streams
- With this type of integration, AWS will manage the poller on your behalf and perform synchronous invocations of your function.

Lambda execution environment

- Lambda invokes your function in an execution environment, which is a secure and isolated environment.
- The execution environment manages the resources required to run your function. The execution environment also provides lifecycle support for the function's runtime and any external extensions associated with your function.



Lambda Cold and warm starts

- A cold start occurs when a new execution environment is required to run a Lambda function. When the Lambda service receives a request to run a function, the service first prepares an execution environment. During this step, the service downloads the code for the function, then creates the execution environment with the specified memory, runtime, and configuration. Once complete, Lambda runs any initialization code outside of the event handler before finally running the handler code.

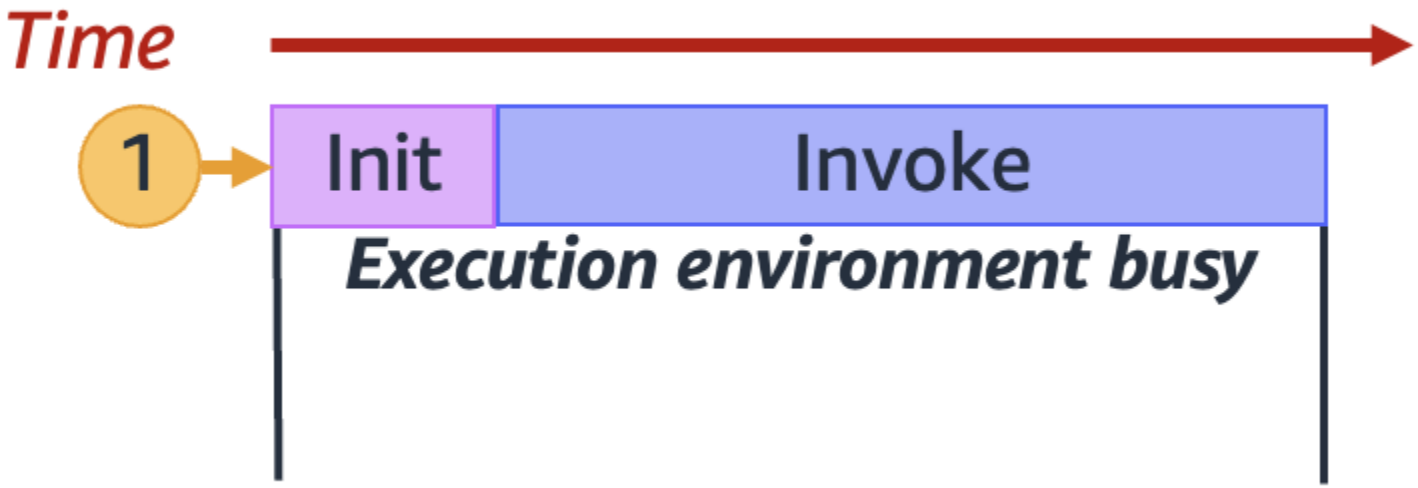
- In a warm start, the Lambda service retains the environment instead of destroying it immediately. This allows the function to run again within the same execution environment. This saves time by not needing to initialize the environment.

Lambda Cold and warm starts

- When you invoke a Lambda function, the invocation is routed to an execution environment to process the request. If the environment is not already initialized, the start-up time of the environment adds to latency. If a function has not been used for some time, if more concurrent invocations are required, or if you update a function, new environments are created. Creation of these environments can introduce latency for the invocations that are routed to a new environment.
- Concurrency is the number of in-flight requests that your AWS Lambda function is handling at the same time. For each concurrent request, Lambda provisions a separate instance of your execution environment. As your functions receive more requests, Lambda automatically handles scaling the number of execution environments until you reach your account's concurrency limit. By default, Lambda provides your account with a total concurrency limit of 1,000 concurrent executions across all functions in an AWS Region.

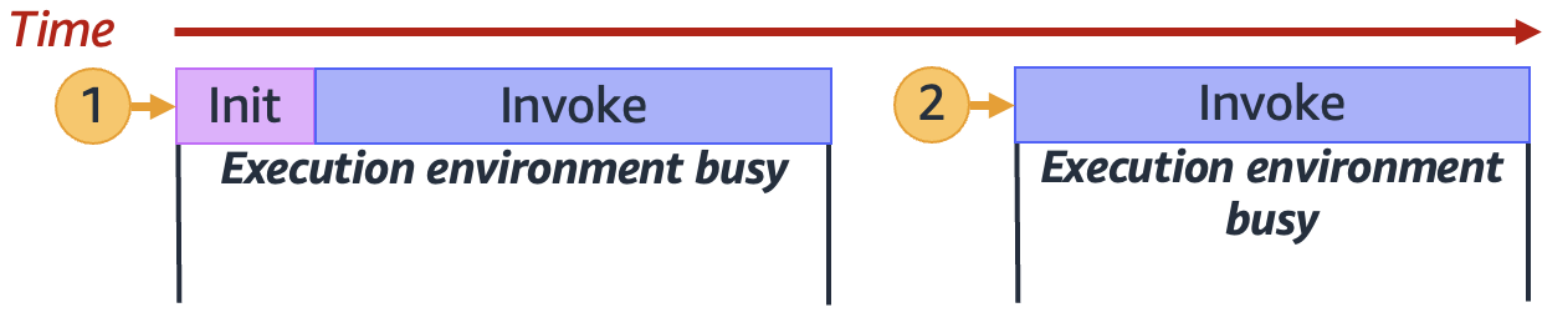
Understanding and visualizing concurrency

- Lambda invokes your function in a secure and isolated execution environment. To handle a request, Lambda must first initialize an execution environment (the Init phase), before using it to invoke your function (the Invoke phase):



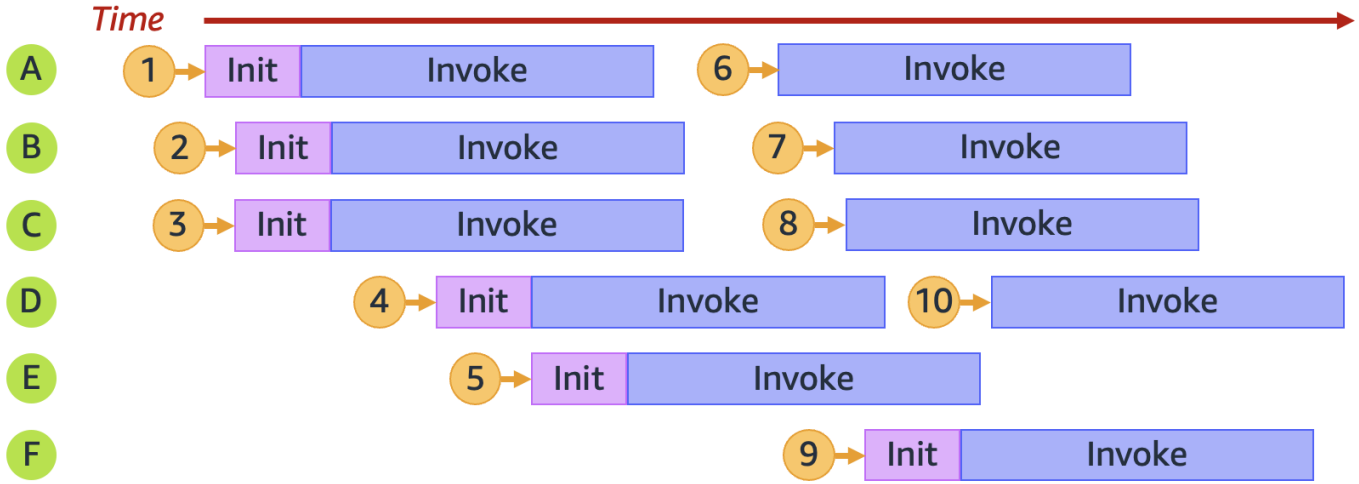
Understanding and visualizing concurrency

- The previous diagram uses a rectangle to represent a single execution environment. When your function receives its very first request (represented by the yellow circle with label 1), Lambda creates a new execution environment and runs the code outside your main handler during the Init phase. Then, Lambda runs your function's main handler code during the Invoke phase. During this entire process, this execution environment is busy and cannot process other requests.
- When Lambda finishes processing the first request, this execution environment can then process additional requests for the same function. For subsequent requests, Lambda doesn't need to re-initialize the environment.
- Lambda reuses the execution environment to handle the second request (represented by the yellow circle with label 2).



Understanding and visualizing concurrency

- So far, we've focused on just a single instance of your execution environment (that is, a concurrency of 1). In practice, Lambda may need to provision multiple execution environment instances in parallel to handle all incoming requests. When your function receives a new request, one of two things can happen:
- If a pre-initialized execution environment instance is available, Lambda uses it to process the request.
- Otherwise, Lambda creates a new execution environment instance to process the request.



Understanding reserved concurrency and provisioned concurrency

- By default, your account has a concurrency limit of 1,000 concurrent executions across all functions in a Region. Your functions share this pool of 1,000 concurrency on an on-demand basis. Your functions experiences throttling (that is, they start to drop requests) if you run out of available concurrency.
- Some of your functions might be more critical than others. As a result, you might want to configure concurrency settings to ensure that critical functions get the concurrency that they need. There are two types of concurrency controls available: reserved concurrency and provisioned concurrency.
- Use reserved concurrency to set both the maximum and minimum number of concurrent instances to reserve a portion of your account's concurrency for a function. This is useful if you don't want other functions taking up all the available unreserved concurrency. When a function has reserved concurrency, no other function can use that concurrency.
- Use provisioned concurrency to pre-initialize a number of environment instances for a function. This is useful for reducing cold start latencies.

Example applications

File Processing

- PDF Encryption Application: Create a serverless application that encrypts PDF files when they are uploaded to an Amazon Simple Storage Service bucket and saves them to another bucket, which is useful for securing sensitive documents upon upload.
- Image Analysis Application: Create a serverless application that extracts text from images using Amazon Rekognition, which is useful for document processing, content moderation, and automated image analysis.

Database Integration

- Queue-to-Database Application: Create a serverless application that writes queue messages to an Amazon RDS database, which is useful for processing user registrations and handling order submissions.

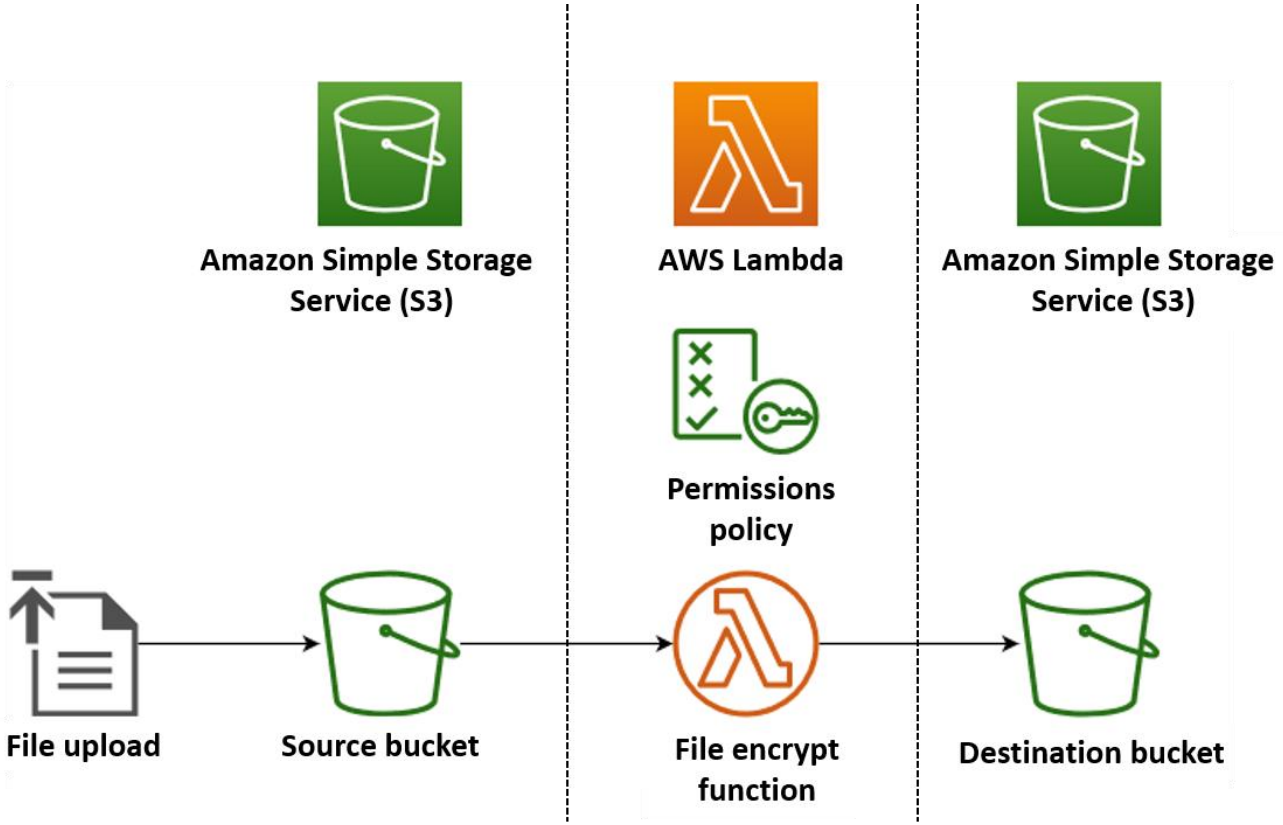
Scheduled Tasks

- Database Maintenance Application: Create a serverless application that automatically deletes entries more than 12 months old from a

Create a serverless file-processing app

In this example, you create an app which automatically encrypts PDF files when they are uploaded to an Amazon Simple Storage Service (Amazon S3) bucket. To implement this app, you create the following resources:

- An S3 bucket for users to upload PDF files to
- A Lambda function in Python which reads the uploaded file and creates an encrypted, password-protected version of it
- A second S3 bucket for Lambda to save the encrypted file in



AWS Identity and Access Management (IAM)

- AWS Identity and Access Management (IAM) is a web service that helps you securely control access to AWS resources. With IAM, you can manage permissions that control which AWS resources users can access. You use IAM to control who is authenticated (signed in) and authorized (has permissions) to use resources.

Identities

- When you create an AWS account, you begin with one sign-in identity called the AWS account root user that has complete access to all AWS services and resources. We strongly recommend that you don't use the root user for everyday tasks
- Use IAM to set up other identities in addition to your root user, such as administrators, analysts, and developers, and grant them access to the resources they need to succeed in their tasks.

AWS Identity and Access Management (IAM)

Access management

- After a user is set up in IAM, they use their sign-in credentials to authenticate with AWS.
- Authentication is provided by matching the sign-in credentials to a principal (an IAM user, AWS STS federated principal, IAM role, or application) trusted by the AWS account.
- Next, a request is made to grant the principal access to resources.
- Access is granted in response to an authorization request if the user has been given permission to the resource. For example, when you first sign in to the console and are on the console Home page, you aren't accessing a specific service. When you select a service, the request for authorization is sent to that service and it looks to see if your identity is on the list of authorized users, what policies are being enforced to control the level of access granted, and any other policies that might be in effect.

AWS Storage

Amazon S3

- Amazon Simple Storage Service (Amazon S3) is an object storage service that offers industry-leading scalability, data availability, security, and performance.

Storage classes

Amazon S3 offers a range of storage classes designed for different use cases. For example, you can store mission-critical production data in S3 Standard or S3 Express One Zone for frequent access, save costs by storing infrequently accessed data in S3 Standard-IA or S3 One Zone-IA, and archive data at the lowest costs in S3 Glacier Instant Retrieval, S3 Glacier Flexible Retrieval, and S3 Glacier Deep Archive.

- Amazon S3 is an object storage service that stores data as objects, hierarchical data, or tabular data within buckets. An object is a file and any metadata that describes the file. A bucket is a container for objects.
- To store your data in Amazon S3, you first create a bucket and specify a bucket name and AWS Region. Then, you upload your data to that bucket as objects in Amazon S3. Each object has a key (or key name), which is the unique identifier for the object within the bucket.

Amazon S3

Amazon S3 Standard (S3 Standard)

- S3 Standard offers high durability, availability, and performance object storage for frequently accessed data. Because it delivers low latency and high throughput, S3 Standard is appropriate for a wide variety of use cases, including cloud applications, dynamic websites, content distribution, mobile and gaming applications, and big data analytics.

Key features:

- General purpose storage for frequently accessed data
- Low latency and high throughput performance
- Designed to deliver 99.99% availability with an availability SLA of 99.9%

Amazon S3

Amazon S3 Intelligent-Tiering (S3 Intelligent-Tiering)

Amazon S3 Intelligent-Tiering (S3 Intelligent-Tiering) is the first cloud storage that automatically reduces your storage costs on a granular object level by automatically moving data to the most cost-effective access tier based on access frequency, without performance impact, retrieval fees, or operational overhead.

S3 Intelligent-Tiering delivers milliseconds latency and high throughput performance for frequently, infrequently, and rarely accessed data in the Frequent, Infrequent, and Archive Instant Access tiers.

For a small monthly object monitoring and automation charge, S3 Intelligent-Tiering monitors access patterns and automatically moves objects that have not been accessed to lower-cost access tiers. S3 Intelligent-Tiering automatically stores objects in three access tiers: one tier that is optimized for frequent access, a 40% lower-cost tier that is optimized for infrequent access, and a 68% lower-cost tier optimized for rarely accessed data.

Amazon S3

Amazon S3 Express One Zone - Optimized for speed, not durability (single AZ)

is a high-performance, single-Availability Zone storage class purpose-built to deliver consistent single-digit millisecond data access for your most frequently accessed data and latency-sensitive applications.

S3 Express One Zone can improve data access speeds by 10x and reduce request costs by up to 80% compared to S3 Standard. While you have always been able to choose a specific AWS Region to store your S3 data, with S3 Express One Zone you can select a specific AWS Availability Zone within an AWS Region to store your data.

You can choose to co-locate your storage and compute resources in the same Availability Zone to further optimize performance, which helps lower compute costs and run workloads faster. With S3 Express One Zone, data is stored in a different bucket type—an Amazon S3 directory bucket—which can support up to 2 million requests per second.

Amazon S3

Amazon S3 Standard-Infrequent Access (S3 Standard-IA)

- S3 Standard-IA is for data that is accessed less frequently, but requires rapid access when needed.
- S3 Standard-IA offers the high durability, high throughput, and low latency of S3 Standard, with a low per GB storage price and per GB retrieval charge.
- This combination of low cost and high performance make S3 Standard-IA ideal for long-term storage, backups, and as a data store for disaster recovery files.
- You can configure S3 storage classes at the object level, and a single bucket can contain objects stored across S3 Standard, S3 Intelligent-Tiering, S3 Standard-IA, and S3 One Zone-IA.
- You can also use S3 Lifecycle policies to automatically transition objects between storage classes without any application changes.

Amazon S3

The Amazon S3 Glacier storage classes

Purpose-built for data archiving, and are designed to provide you with the highest performance, the most retrieval flexibility, and the lowest cost archive storage in the cloud. You can choose from three archive storage classes optimized for different access patterns and storage duration. For archive data that needs immediate access, such as medical images, news media assets, or genomics data, choose the S3 Glacier Instant Retrieval storage class, an archive storage class that delivers the lowest cost storage with milliseconds retrieval. For archive data that does not require immediate access but needs the flexibility to retrieve large sets of data at no cost, such as backup or disaster recovery use cases, choose S3 Glacier Flexible Retrieval (formerly S3 Glacier), with retrieval in minutes or free bulk retrievals in 5—12 hours. To save even more on long-lived archive storage such as compliance archives and digital media preservation, choose S3 Glacier Deep Archive, the lowest cost storage in the cloud with data retrieval from 12—48 hours.

RDS

Amazon Relational Database Service

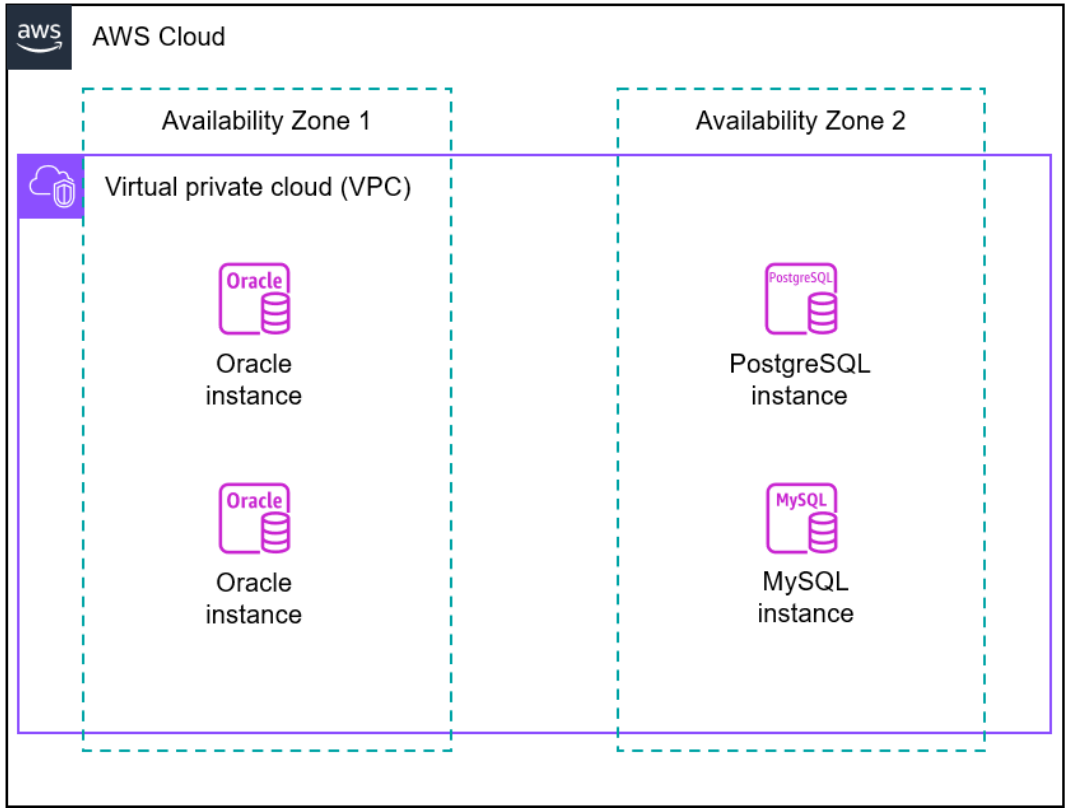
- Amazon Relational Database Service (Amazon RDS) is a web service that makes it easier to set up, operate, and scale a relational database in the AWS Cloud
- Amazon RDS provides the following principal advantages over database deployments that aren't fully managed:
  - You can use database engines that you are already familiar with: IBM Db2, MariaDB, Microsoft SQL Server, MySQL, Oracle Database, and PostgreSQL.
  - Amazon RDS manages backups, software patching, automatic failure detection, and recovery.
  - You can turn on automated backups, or manually create your own backup snapshots. You can use these backups to restore a database. The Amazon RDS restore process works reliably and efficiently.
  - You can get high availability with a primary DB instance and a synchronous secondary DB instance that you can fail over to when problems occur. You can also use read replicas to increase read scaling.
  - In addition to the security in your database package, you can control access by using AWS Identity and Access Management (IAM) to define users and permissions. You can also help protect your databases by putting them in a virtual private cloud (VPC).

Amazon Relational Database Service

Feature	On-premises management	Amazon EC2 management	Amazon RDS management
Application optimization	Customer	Customer	Customer
Scaling	Customer	Customer	AWS
High availability	Customer	Customer	AWS
Database backups	Customer	Customer	AWS
Database software patching	Customer	Customer	AWS
Database software install	Customer	Customer	AWS
Operating system (OS) patching	Customer	Customer	AWS
OS installation	Customer	Customer	AWS
Server maintenance	Customer	AWS	AWS
Hardware lifecycle	Customer	AWS	AWS
Power, network, and cooling	Customer	AWS	AWS

Amazon Relational Database Service

- A DB instance is an isolated database environment in the AWS Cloud. The basic building block of Amazon RDS is the DB instance. Your DB instance can contain one or more user-created databases. The following diagram shows a virtual private cloud (VPC) that contains two Availability Zones, with each AZ containing two DB instances.

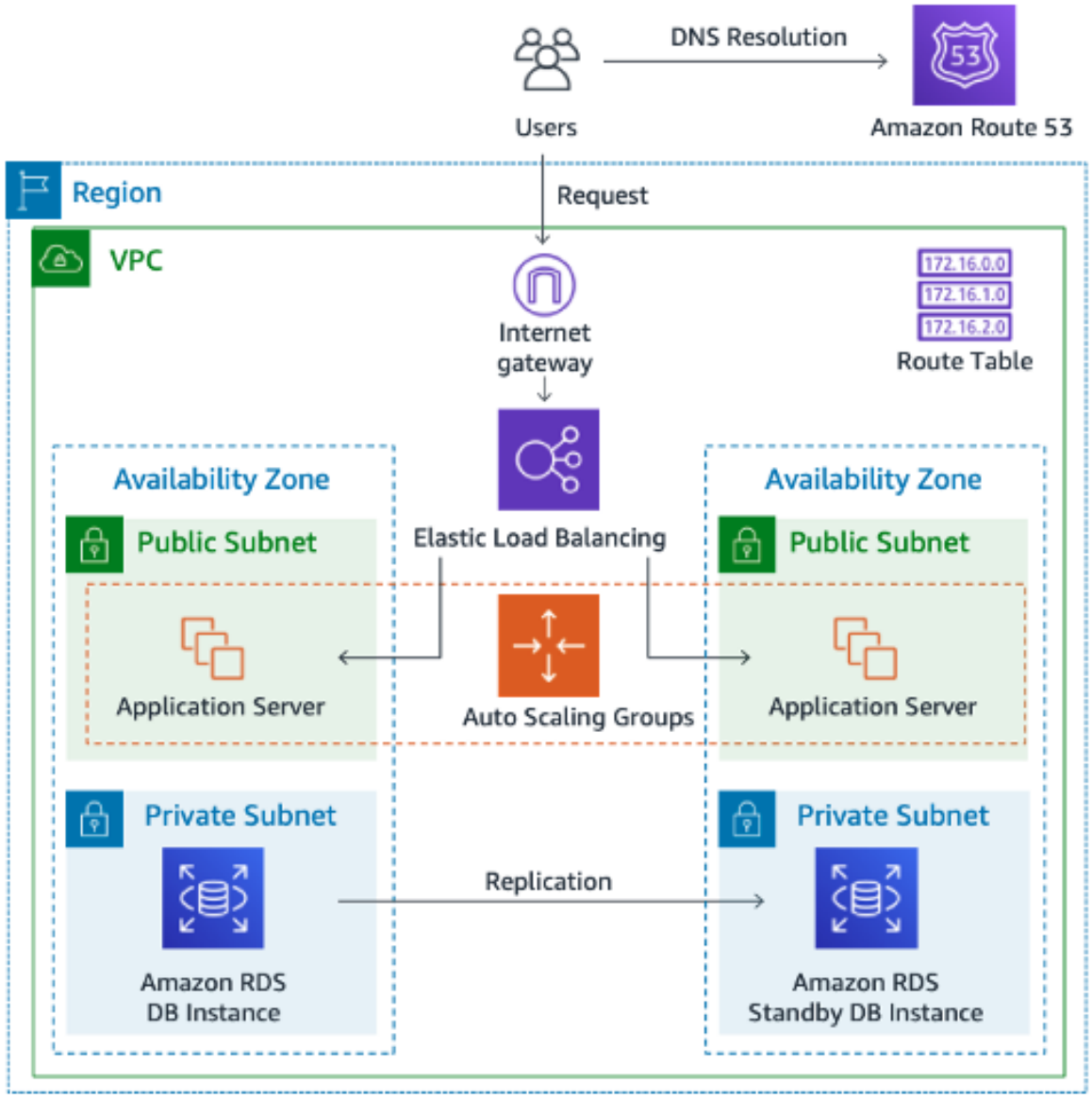


Amazon Relational Database Service

The following image shows a typical use case of a dynamic website that uses Amazon RDS DB instances for database storage

The EC2 application servers interact with RDS DB instances. The DB instances reside in private subnets within different Availability Zones (AZs) within the same Virtual Private Cloud (VPC). Because the subnets are private, no requests from the internet are permitted.

The primary DB instance replicates to another DB instance, called a read replica. Both DB instances are in private subnets within the VPC, which means that Internet users can't access them directly.



Amazon Relational Database Service

A DB engine is the specific relational database software that runs on your DB instance. Amazon RDS supports the following database engines:

- IBM Db2
- MariaDB
- Microsoft SQL Server
- MySQL
- Oracle Database
- PostgreSQL

Amazon Relational Database Service

Amazon EBS provides durable, block-level storage volumes that you can attach to a running instance. DB instance storage comes in the following types:

- General Purpose (SSD):This cost-effective storage type is ideal for a broad range of workloads running on medium-sized DB instances. General Purpose storage is best suited for development and testing environments.
- Provisioned IOPS (PIOPS): This storage type is designed to meet the needs of I/O-intensive workloads, particularly database workloads, that require low I/O latency and consistent I/O throughput. Provisioned IOPS storage is best suited for production environments.

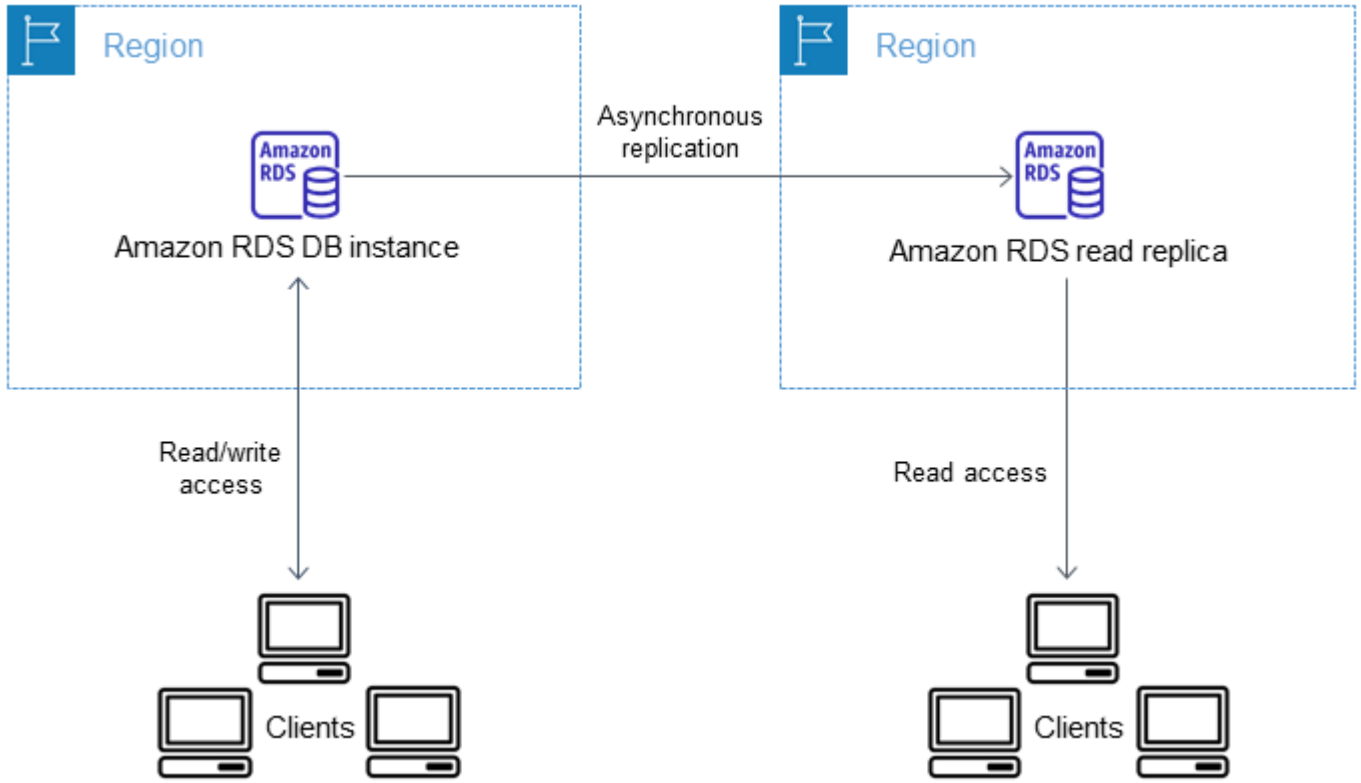
The storage types differ in performance characteristics and price. You can tailor your storage performance and cost to the requirements of your database.

Each DB instance has minimum and maximum storage requirements depending on the storage type and the database engine it supports. It's important to have sufficient storage so that your databases have room to grow.

Amazon Relational Database Service

Amazon cloud computing resources are housed in highly available data center facilities in different areas of the world (for example, North America, Europe, or Asia). Each data center location is called an AWS Region. With Amazon RDS, you can create your DB instances in multiple Regions.

The following scenario shows an RDS DB instance in one Region that replicates asynchronously to a standby DB instance in a different Region. If one Region becomes unavailable, the instance in the other Region is still available.



Amazon Relational Database Service

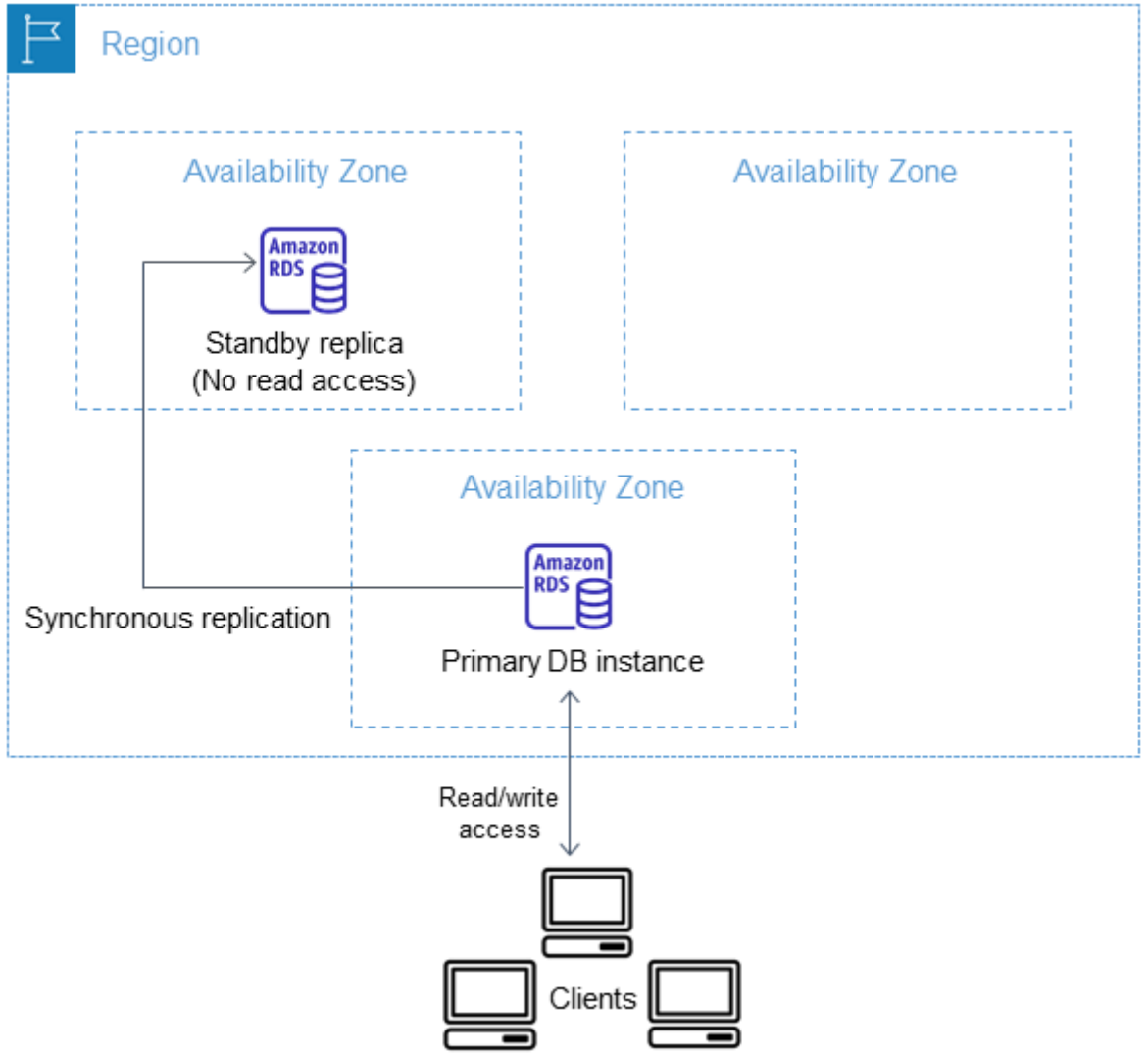
You can run your DB instance in several Availability Zones, an option called a Multi-AZ deployment. When you choose this option, Amazon automatically provisions and maintains one or more secondary standby DB instances in a different AZ. Your primary DB instance is replicated across Availability Zones to each secondary DB instance.

A Multi-AZ deployment provides the following advantages:

- Providing data redundancy and failover support
- Eliminating I/O freezes
- Minimizing latency spikes during system backups
- Serving read traffic on secondary DB instances (Multi-AZ DB clusters deployment only)

Amazon Relational Database Service

The following diagram depicts a Multi-AZ DB instance deployment, where Amazon RDS automatically provisions and maintains a synchronous standby replica in a different Availability Zone. The replica database doesn't serve read traffic.



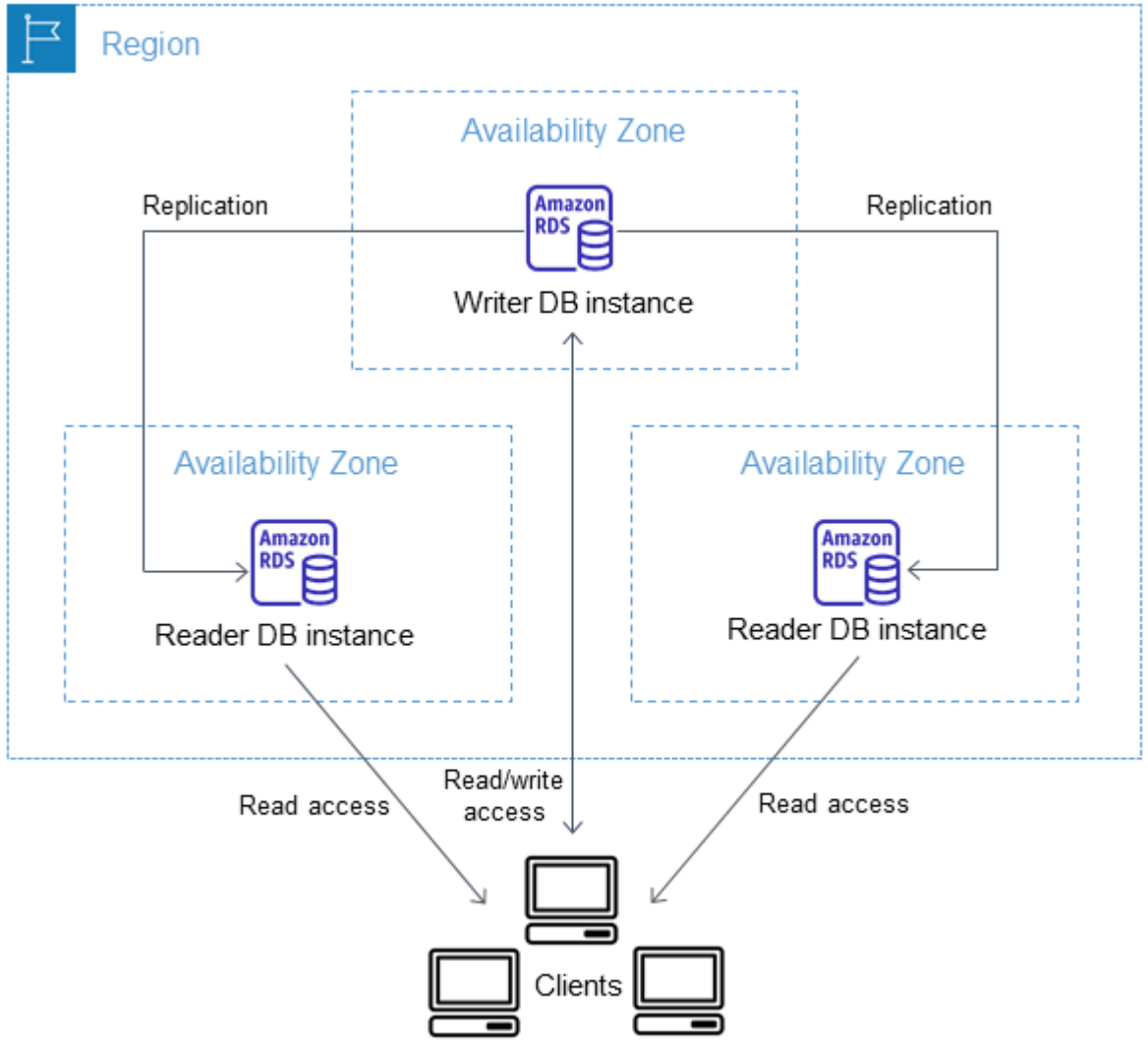
Amazon Relational Database Service

The following diagram depicts a Multi-AZ DB cluster deployment, which has a writer DB instance and two reader DB instances in three separate Availability Zones in the same AWS Region. All three DB instances can serve read traffic.

Read replicas exist to scale reads and offload work from the primary database.

A single database can handle only so many read queries. Read replicas: Receive read-only copies of data from the primary DB, Allow applications to send SELECT queries to replicas, Reduce load on the primary (writer) instance

Read replicas can:Be promoted to standalone DBs. Act as a manual recovery option



Amazon Relational Database Service

Synchronous Replication

- Client sends a write request
- Primary writes data locally
- Primary sends the data to the replica
- Replica confirms the write
- Only then does the primary acknowledge success to the client

Asynchronous Replication

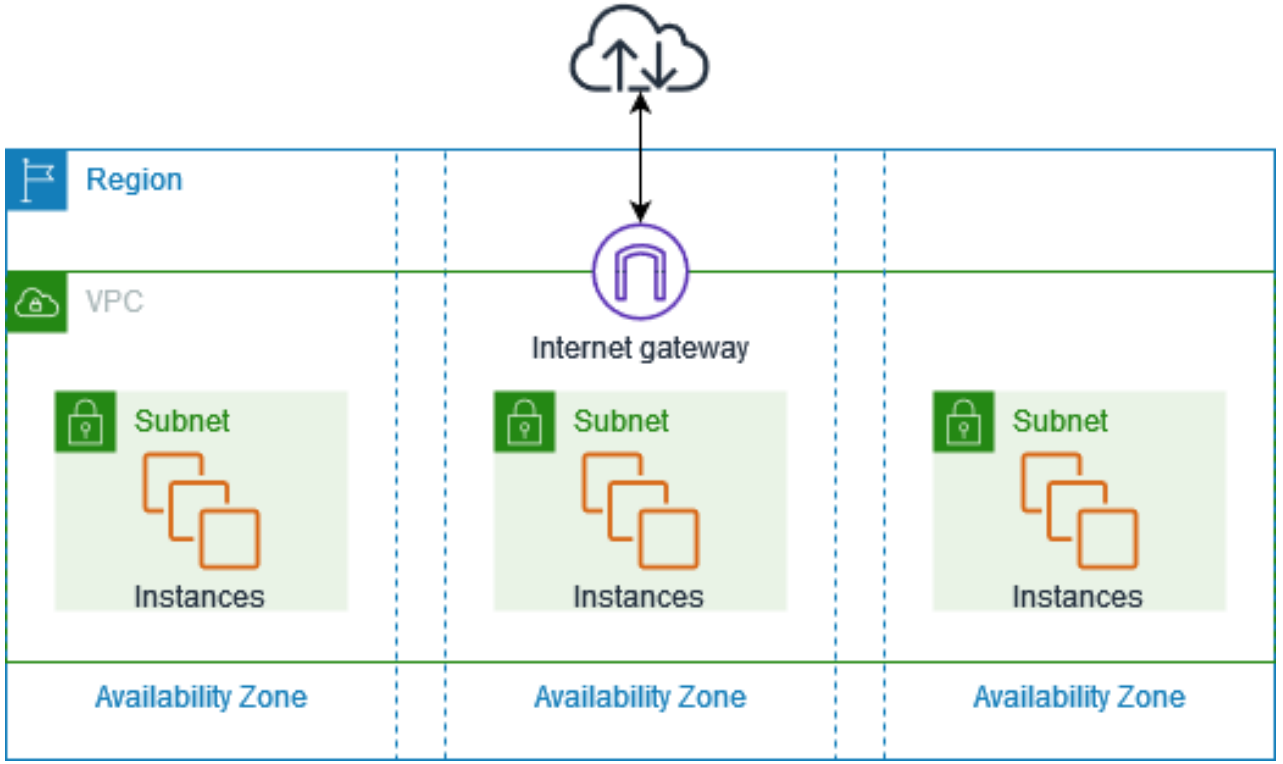
- Client sends a write
- Primary writes data locally
- Primary immediately responds to client
- Data is sent to replica later

AWS Networking VPC

VPC - Virtual Private Cloud (Amazon VPC)

With Amazon Virtual Private Cloud (Amazon VPC), you can launch AWS resources in a logically isolated virtual network that you've defined. This virtual network closely resembles a traditional network that you'd operate in your own data center, with the benefits of using the scalable infrastructure of AWS.

The following diagram shows an example VPC. The VPC has one subnet in each of the Availability Zones in the Region, EC2 instances in each subnet, and an internet gateway to allow communication between the resources in your VPC and the internet.



Understanding reserved concurrency and provisioned concurrency

- Your AWS account contains a default VPC for each AWS Region. This default VPC comes pre-configured with settings that make it a convenient option for quickly launching resources. However, the default VPC may not always align with your long-term networking needs. This is where creating additional VPCs can be advantageous.
- Creating additional VPCs offers several advantages over relying on the default VPC that comes provisioned with every new AWS account. With a self-managed VPC, you can architect the network topology to align precisely with your specific requirements, whether that's implementing a multi-tier application, connecting to on-premises resources, or segregating workloads by department or business unit.
- In addition, creating multiple VPCs can enable greater security and isolation between your different applications or business units. Each VPC acts as a separate, virtual network, allowing you to apply distinct security policies, access controls, and routing configurations tailored to each environment.

VPC elements

VPC:

- Logical network boundary
- Defined by a CIDR block (IPv4 / IPv6)
- One VPC per region (many allowed)

CIDR block:

- Private IP address range
- Example:10.0.0.0/16

Subnets

- Subdivisions of the VPC CIDR
- Exist in one Availability Zone
- Can be: Public or Private

Route Tables

- Control where traffic is sent
- Associated with subnets
- Contain: local route + Optional IGW / NAT / TGW routes

Internet Gateway (IGW)

- Enables internet access
- Attached to the VPC
- Required for public subnets

NAT Gateway:

- Outbound internet for private subnets, Uses Elastic IP
- No inbound connections

VPC elements

VPC Peering

- Direct connection between two VPCs

Non-transitive

Transit Gateway (TGW)

- Central routing hub
- Connects: Multiple VPCs

Security Groups

- Stateful firewall
- Applied to ENIs / instances
- Allow rules only
- Network ACLs (NACLs)

Stateless firewall

- Applied at subnet level
- Allow + deny rules

VPC basics - Subnets

- A VPC spans all of the Availability Zones in a Region. After you create a VPC, you can add one or more subnets in each Availability Zone.

Availability Zones

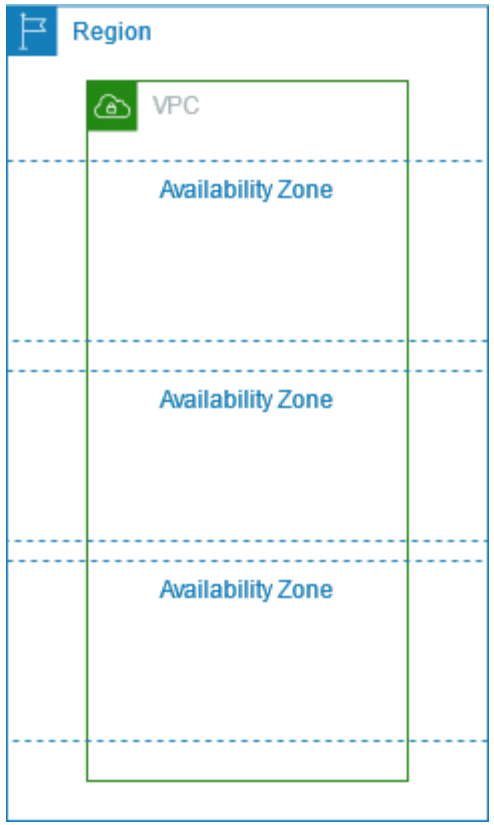
- Discrete data centers with redundant power, networking, and connectivity in an AWS Region. You can use multiple AZs to operate production applications and databases that are more highly available, fault tolerant, and scalable than would be possible from a single data center. If you partition your applications running in subnets across AZs, you are better isolated and protected from issues such as power outages, lightning strikes, tornadoes, and earthquakes

CIDR blocks

You must specify IP address ranges for your VPC and subnets.

Subnets

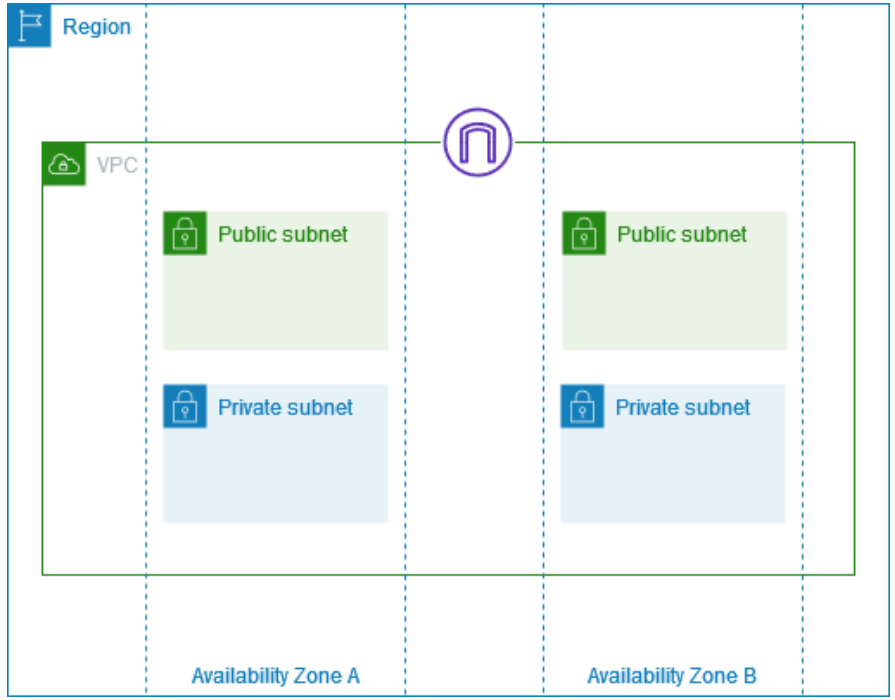
- A range of IP addresses in your VPC. You can launch AWS resources, such as EC2 instances, into your subnets. Each subnet resides entirely within one Availability Zone



VPC basics - Subnets

The subnet type is determined by how you configure routing for your subnets. For example:

- Public subnet – The subnet has a direct route to an internet gateway. Resources in a public subnet can access the public internet.
- Private subnet – The subnet does not have a direct route to an internet gateway. Resources in a private subnet require a NAT device to access the public internet.
- VPN-only subnet – The subnet has a route to a Site-to-Site VPN connection through a virtual private gateway. The subnet does not have a route to an internet gateway.
- Isolated subnet – The subnet has no routes to destinations outside its VPC. Resources in an isolated subnet can only access or be accessed by other resources in the same VPC.



VPC basics – Subnets - Private IPv4 addresses

- Private IPv4 addresses (also referred to as private IP addresses in this topic) are not reachable over the internet, and can be used for communication between the instances in your VPC.
- When you launch an instance into a VPC, a primary private IP address from the IPv4 address range of the subnet is assigned to the primary network interface (for example, eth0) of the instance.
- Each instance is also given a private (internal) DNS hostname that resolves to the private IP address of the instance.

VPC basics – Subnets - Public IPv4 addresses

- All subnets have an attribute that determines whether a network interface created in the subnet automatically receives a public IPv4 address (also referred to as a public IP address in this topic).
- Therefore, when you launch an instance into a subnet that has this attribute enabled, a public IP address is assigned to the primary network interface that's created for the instance.
- A public IP address is mapped to the primary private IP address through network address translation (NAT).

You can control whether your instance receives a public IP address by doing the following:

- Modifying the public IP addressing attribute of your subnet.
- Enabling or disabling the public IP addressing feature during instance launch, which overrides the subnet's public IP addressing attribute.
- You can unassign a public IP address from your instance after launch by managing the IP addresses associated with a network interface.

VPC basics - Subnets

Key characteristics of a public subnet

- Internet connectivity (via Internet Gateway)
- Resources can be reachable from the internet if:
- They have a public IPv4 address or Elastic IP
- Their security group and NACL allow inbound traffic

Commonly hosts:

- EC2 instances acting as web servers
- Load balancers (ALB / NLB)
- NAT Gateways

What makes it public

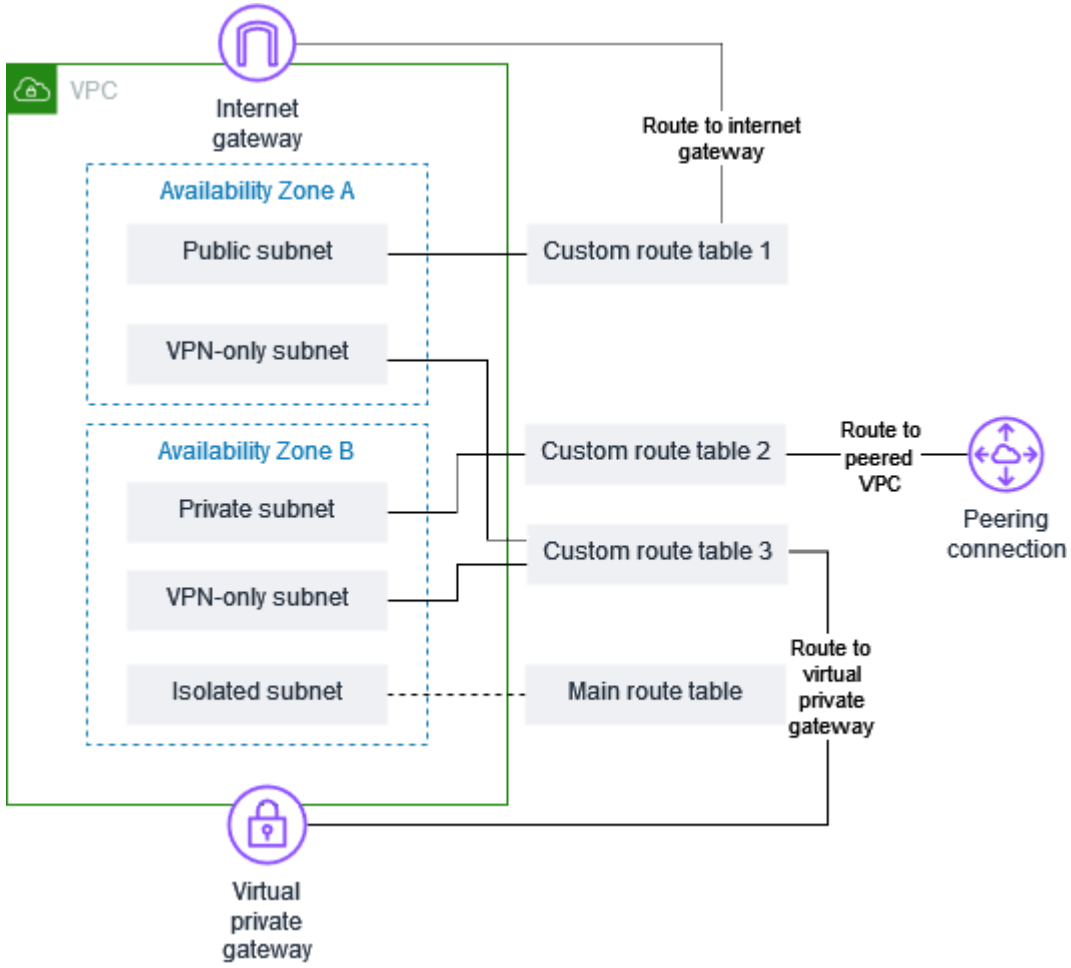
It is not the instance that makes a subnet public —It is the route table.

VPC basics – route tables

- A route table serves as the traffic controller for your virtual private cloud (VPC).
- Each route table contains a set of rules, called routes, that determine where network traffic from your subnet or gateway is directed.
- When you create a VPC, we also create the main route table for the VPC.
- You can create additional route tables for your VPC, so that you have more granular control over the network paths for your VPC.
- Main route table—The route table that automatically comes with your VPC. It controls the routing for all subnets that are not explicitly associated with any other route table.
- Custom route table—A route table that you create for your VPC.
- Gateway route table—A route table that's associated with an internet gateway or virtual private gateway.

VPC basics – Example VPC with route tables

- The following diagram shows a VPC with five subnets, a main route table, and three custom route tables.
- All four route tables have local routes.
- Custom route table 1 has a route to an internet gateway, and it is associated with the public subnet in Availability Zone A.
- Custom route table 2 has a route to a peered VPC, and it is associated with the private subnet in Availability Zone B.
- Custom route table 3 has a route to a virtual private gateway, and it is associated with the VPN-only subnets in both Availability Zones



VPC basics – Route tables

Main Route table

- When you create a VPC, it automatically has a main route table. When a subnet does not have an explicit routing table associated with it, the main routing table is used by default.
- By default, when you create a non-default VPC, the main route table contains only a local route. If you Create a VPC and choose a NAT gateway, Amazon VPC automatically adds routes to the main route table for the gateways.

Custom route table

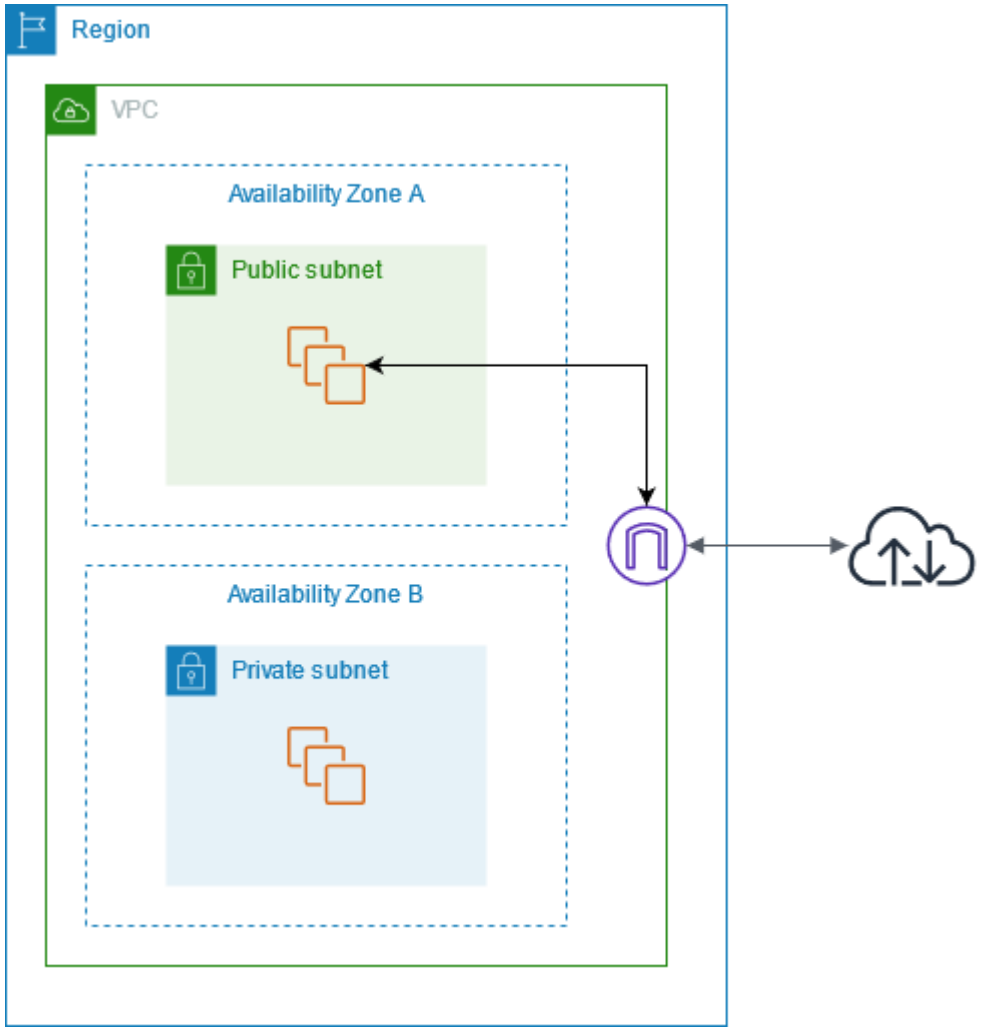
- By default, a route table contains a local route for communication within the VPC. If you Create a VPC and choose a public subnet, Amazon VPC creates a custom route table and adds a route that points to the internet gateway.
- Each subnet in your VPC must be associated with a route table. A subnet can be explicitly associated with custom route table, or implicitly or explicitly associated with the main route table.

VPC basics – Internet Gateway

- An internet gateway is a horizontally scaled, redundant, and highly available VPC component that allows communication between your VPC and the internet. It supports IPv4 and IPv6 traffic. It does not cause availability risks or bandwidth constraints on your network traffic.
- An internet gateway enables resources in your public subnets (such as EC2 instances) to connect to the internet if the resource has a public IPv4 address or an IPv6 address.
- Similarly, resources on the internet can initiate a connection to resources in your subnet using the public IPv4 address or IPv6 address. For example, an internet gateway enables you to connect to an EC2 instance in AWS using your local computer.
- If a subnet is associated with a route table that has a route to an internet gateway, it's known as a public subnet. If a subnet is associated with a route table that does not have a route to an internet gateway, it's known as a private subnet.

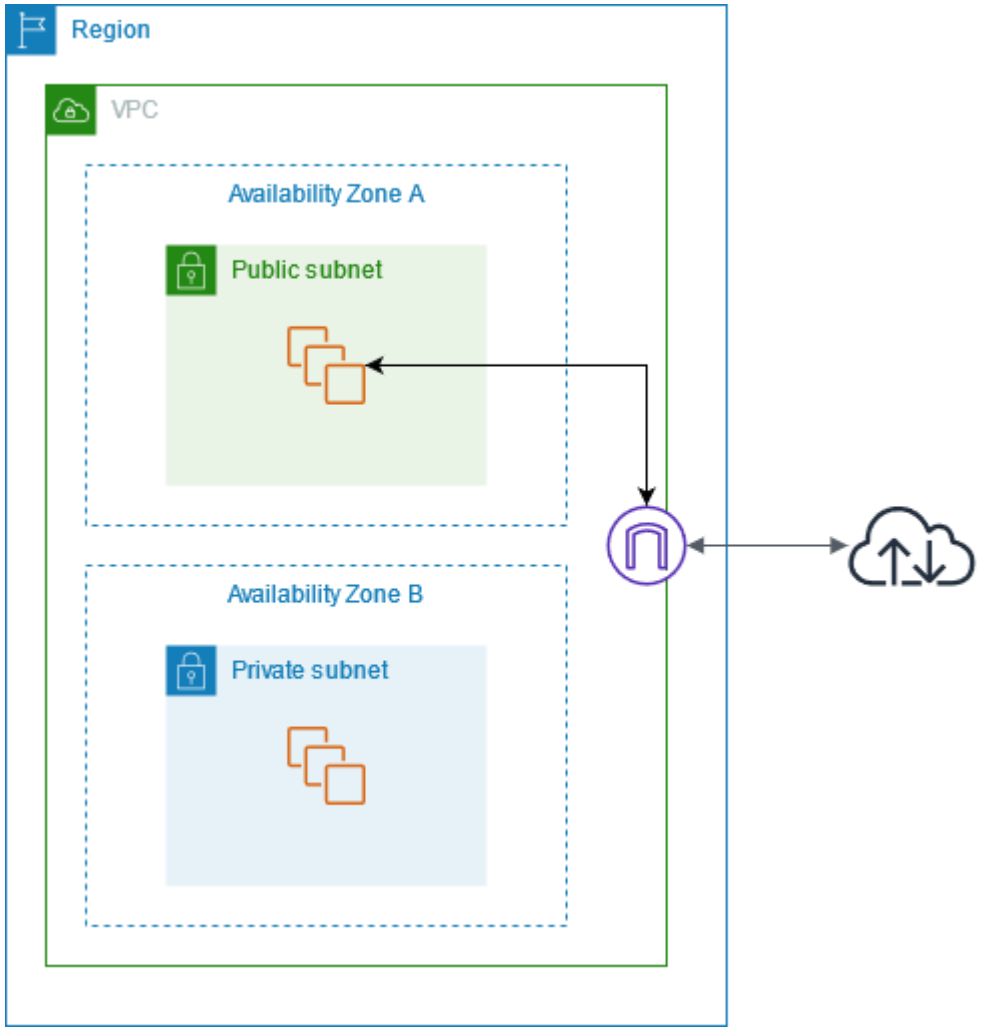
VPC basics – Internet Gateway

In the following diagram, the subnet in Availability Zone A is a public subnet because its route table has a route that sends all internet-bound IPv4 traffic to the internet gateway. The instances in the public subnet must have public IP addresses or Elastic IP addresses to enable communication with the internet over the internet gateway. For comparison, the subnet in Availability Zone B is a private subnet because its route table does not have a route to the internet gateway. Because there is no route to the internet gateway, instances in the private subnet can't communicate with the internet, even if they have public IP addresses.



VPC basics – Internet Gateway

To enable communication over the internet for IPv4, your instance must have a public IPv4 address. You can either configure your VPC to automatically assign public IPv4 addresses to your instances, or you can assign Elastic IP addresses to your instances. Your instance is only aware of the private (internal) IP address space defined within the VPC and subnet. The internet gateway logically provides the one-to-one NAT on behalf of your instance, so that when traffic leaves your VPC subnet and goes to the internet, the reply address field is set to the public IPv4 address or Elastic IP address of your instance, and not its private IP address. Conversely, traffic that's destined for the public IPv4 address or Elastic IP address of your instance has its destination address translated into the instance's private IPv4 address before the traffic is delivered to the VPC.



VPC basics – Subnets - Example

VPC CIDR: 10.0.0.0/16

Subnet CIDR: 10.0.1.0/24

Availability Zone: us-east-1a (example)

Route table attached to the subnet

10.0.0.0/16 → local

0.0.0.0/0 → Internet Gateway

3 EC2 instances with Auto-assign public IPv4 = Enabled.

EC2-A	*10.0.1.10*	*54.210.10.1*
EC2-B	*10.0.1.11*	*18.202.44.9*
EC2-C	*10.0.1.12*	*3.88.155.22*

VPC basics – Subnets - Example

Example: EC2-A calls google.com

EC2-A sends packet:

Source: 10.0.1.10

Destination: 142.250.72.14

EC2 Route table says:

0.0.0.0/0 → IGW

Packet reaches the Internet Gateway

IGW performs 1-to-1 NAT: 10.0.1.10 → 54.210.10.1

Packet goes to the Internet

Response comes back to 54.210.10.1

IGW translates it back to 10.0.1.10

EC2-A receives the response

VPC basics – NAT Gateway

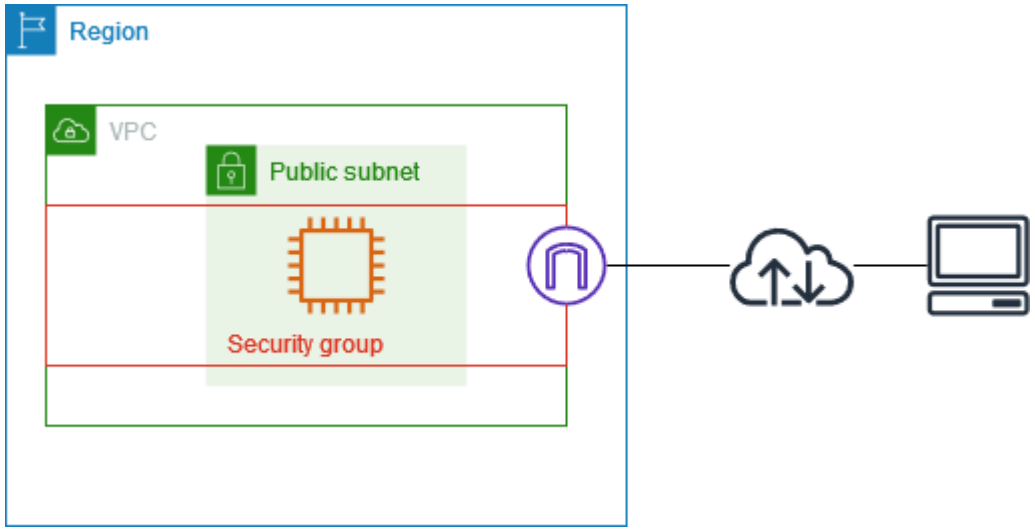
- A NAT gateway is a Network Address Translation (NAT) service. You can use a NAT gateway so that instances in a private subnet can connect to services outside your VPC but external services can't initiate a connection with those instances.
- When you create a NAT gateway, you specify one of the following connectivity types:
  - Public – (Default) Instances in private subnets can connect to the internet through a public NAT gateway, but the instances can't receive unsolicited inbound connections from the internet. You create a public NAT gateway in a public subnet and must associate an Elastic IP address with the NAT gateway at creation. You route traffic from the NAT gateway to the internet gateway for the VPC. Alternatively, you can use a public NAT gateway to connect to other VPCs or your on-premises network. In this case, you route traffic from the NAT gateway through a transit gateway or a virtual private gateway.
  - Private – Instances in private subnets can connect to other VPCs or your on-premises network through a private NAT gateway, but the instances can't receive unsolicited inbound connections from the other VPCs or the on-premises network. You can route traffic from the NAT gateway through a transit gateway or a virtual private gateway. You can't associate an Elastic IP address with a private NAT gateway. You can attach an internet gateway to a VPC with a private NAT gateway, but if you route traffic from the private NAT gateway to the internet gateway, the internet gateway drops the traffic.

VPC basics – Security Groups

- A security group controls the traffic that is allowed to reach and leave the resources that it is associated with. For example, after you associate a security group with an EC2 instance, it controls the inbound and outbound traffic for the instance.
- When you create a VPC, it comes with a default security group. You can create additional security groups for a VPC, each with their own inbound and outbound rules. You can specify the source, port range, and protocol for each inbound rule. You can specify the destination, port range, and protocol for each outbound rule.

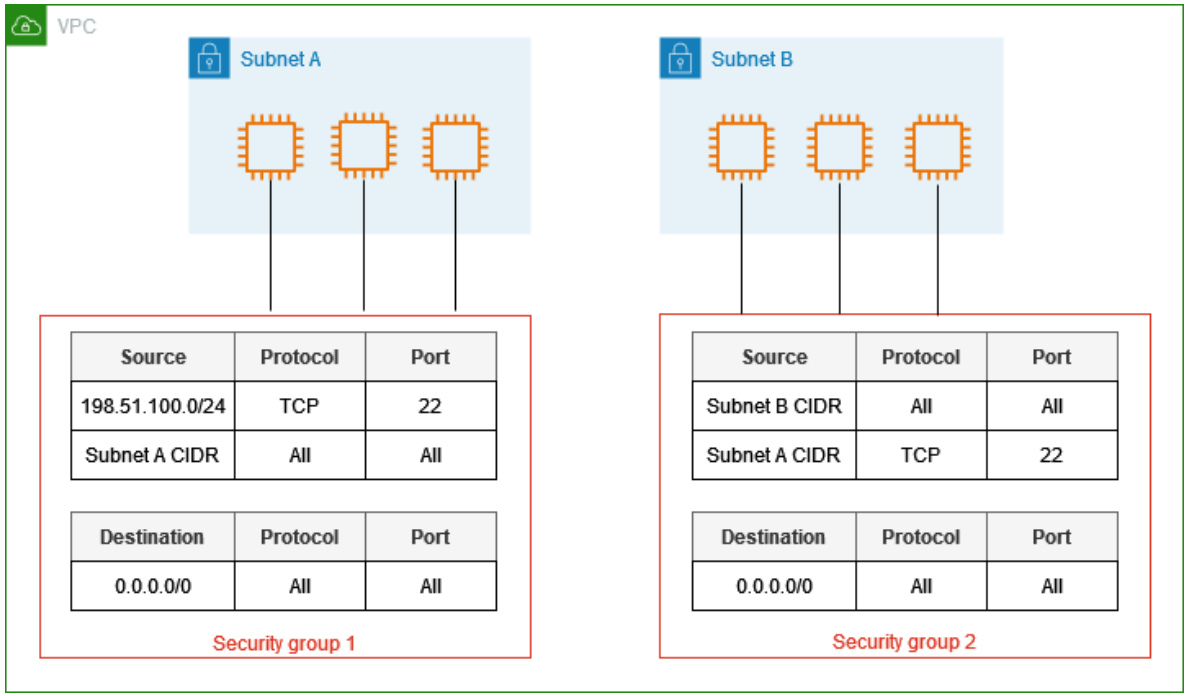
VPC basics – Security Groups

- The following diagram shows a VPC with a subnet, an internet gateway, and a security group. The subnet contains an EC2 instance. The security group is assigned to the instance. The security group acts as a virtual firewall. The only traffic that reaches the instance is the traffic allowed by the security group rules. For example, if the security group contains a rule that allows ICMP traffic to the instance from your network, then you could ping the instance from your computer. If the security group does not contain a rule that allows SSH traffic, then you could not connect to your instance using SSH.



VPC basics – Security Groups

- The following diagram shows a VPC with two security groups and two subnets. The instances in subnet A have the same connectivity requirements, so they are associated with security group 1. The instances in subnet B have the same connectivity requirements, so they are associated with security group 2. The security group rules allow traffic as follows:
  - The first inbound rule in security group 1 allows SSH traffic to the instances in subnet A from the specified address range (for example, a range in your own network).
  - The second inbound rule in security group 1 allows the instances in subnet A to communicate with each other using any protocol and port.
  - The first inbound rule in security group 2 allows the instances in subnet B to communicate with each other using any protocol and port.
  - The second inbound rule in security group 2 allows the instances in subnet A to communicate with the instances in subnet B using SSH.
  - Both security groups use the default outbound rule, which allows all traffic.



VPC basics – Network ACL

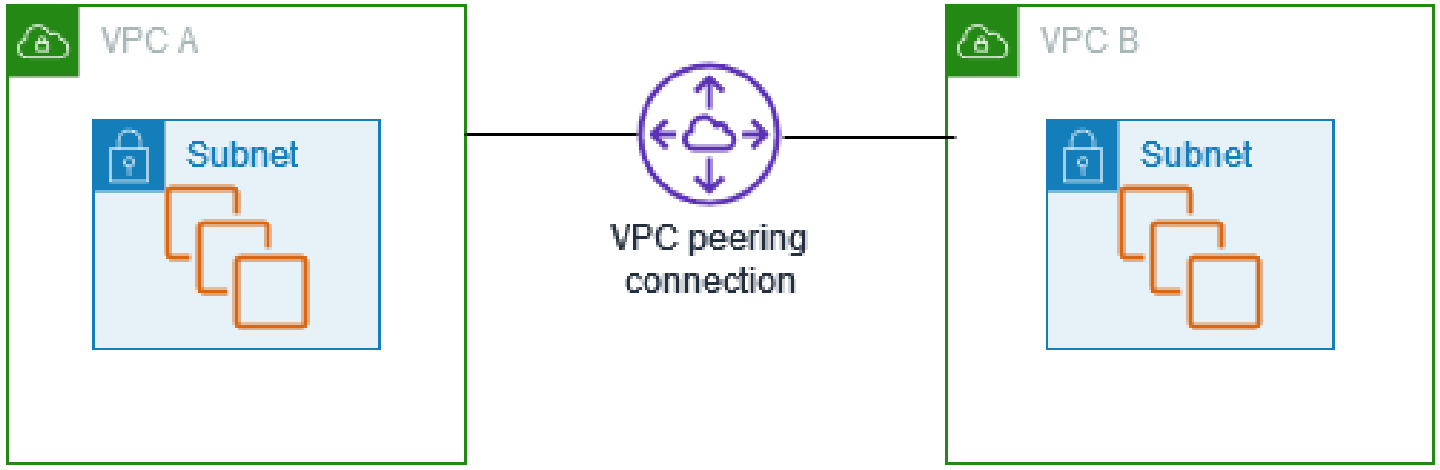
- A network access control list (ACL) allows or denies specific inbound or outbound traffic at the subnet level. You can use the default network ACL for your VPC, or you can create a custom network ACL for your VPC with rules that are similar to the rules for your security groups in order to add an additional layer of security to your VPC.
- Each subnet in your VPC must be associated with a network ACL. If you don't explicitly associate a subnet with a network ACL, the subnet is automatically associated with the default network ACL.
- You can create a custom network ACL and associate it with a subnet to allow or deny specific inbound or outbound traffic at the subnet level.
- You can associate a network ACL with multiple subnets. However, a subnet can be associated with only one network ACL at a time. When you associate a network ACL with a subnet, the previous association is removed.

VPC basics – Network ACL

- NACLs are stateless, which means that information about previously sent or received traffic is not saved. If, for example, you create a NACL rule to allow specific inbound traffic to a subnet, responses to that traffic are not automatically allowed. This is in contrast to how security groups work. Security groups are stateful, which means that information about previously sent or received traffic is saved. If, for example, a security group allows inbound traffic to an EC2 instance, responses are automatically allowed regardless of outbound security group rules.
- **Security Groups protect instances; Network ACLs protect subnets.**
- **Security Groups are the primary control, NACLs are a coarse safety net.**

VPC Peering

- A VPC peering connection is a networking connection between two VPCs that enables you to route traffic between them using private IPv4 addresses or IPv6 addresses. Instances in either VPC can communicate with each other as if they are within the same network. You can create a VPC peering connection between your own VPCs, or with a VPC in another AWS account. The VPCs can be in different Regions (also known as an inter-Region VPC peering connection).



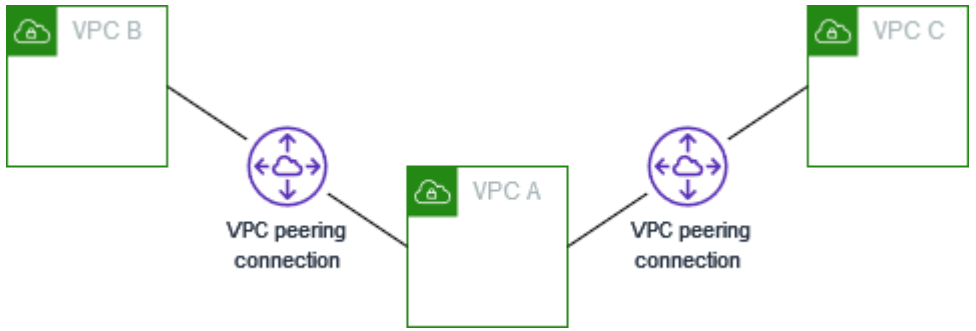
VPC Peering

The following steps describe the VPC peering process:

- The owner of the requester VPC sends a request to the owner of the accepter VPC to create the VPC peering connection. The accepter VPC can be owned by you, or another AWS account, and cannot have a CIDR block that overlaps with the CIDR block of the requester VPC.
- The owner of the accepter VPC accepts the VPC peering connection request to activate the VPC peering connection.
- To enable the flow of traffic between the VPCs using private IP addresses, the owner of each VPC in the VPC peering connection must manually add a route to one or more of their VPC route tables that points to the IP address range of the other VPC (the peer VPC).
- If required, update the security group rules that are associated with your EC2 instance to ensure that traffic to and from the peer VPC is not restricted. If both VPCs are in the same Region, you can reference a security group from the peer VPC as a source or destination for inbound or outbound rules in your security group.

VPC Peering

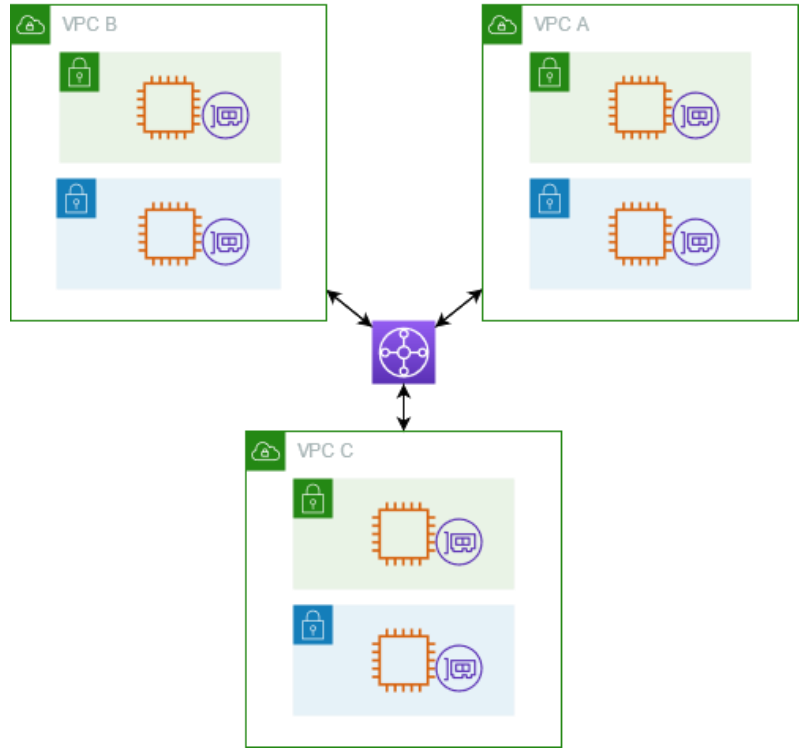
- A VPC peering connection is a one to one relationship between two VPCs. You can create multiple VPC peering connections for each VPC that you own, but transitive peering relationships are not supported. You do not have any peering relationship with VPCs that your VPC is not directly peered with.
- The following diagram is an example of one VPC peered to two different VPCs. There are two VPC peering connections: VPC A is peered with both VPC B and VPC C. VPC B and VPC C are not peered, and you cannot use VPC A as a transit point for peering between VPC B and VPC C. If you want to enable routing of traffic between VPC B and VPC C, you must create a unique VPC peering connection between them.



VPC basics – Transit Gateway

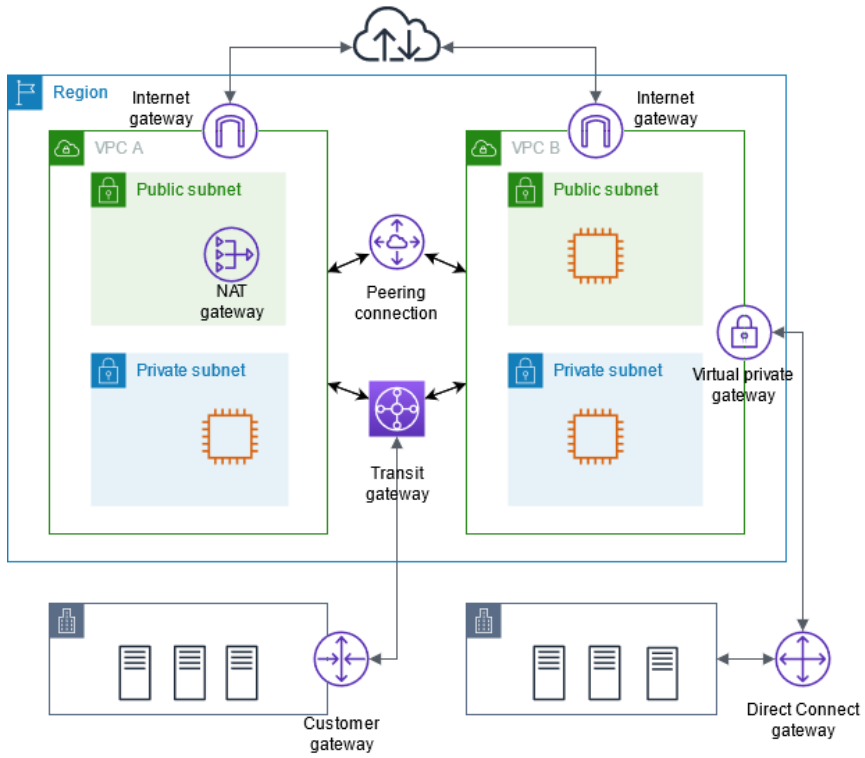
- In AWS Transit Gateway, a transit gateway acts as a Regional virtual router for traffic flowing between your virtual private clouds (VPCs) and on-premises networks.
- The following diagram shows a transit gateway with three VPC attachments. The route table for each of these VPCs includes the local route and routes that send traffic destined for the other two VPCs to the transit gateway.
- Route vs Transit Gateway

A route table decides where traffic should go, while a Transit Gateway actually moves traffic between VPCs and networks.



Connect your VPC to other networks

- You can connect your virtual private cloud (VPC) to other networks, such as other VPCs, the internet, or your on-premises network.
- The diagram demonstrates some of these connectivity options. VPC A is connected to the internet through an internet gateway, and the EC2 instance in the private subnet can connect to the internet using a NAT gateway in the public subnet. VPC B is also connected to the internet, but through a direct internet gateway, allowing the EC2 instance in the public subnet to access the internet.
- Moreover, VPC A and VPC B are connected to each other through a VPC peering connection and a transit gateway. The transit gateway has a VPN attachment to a data center, and VPC B has an Direct Connect connection to the same data center. This interconnectivity enables organizations to integrate their cloud resources with on-premises infrastructure, creating a hybrid cloud environment.



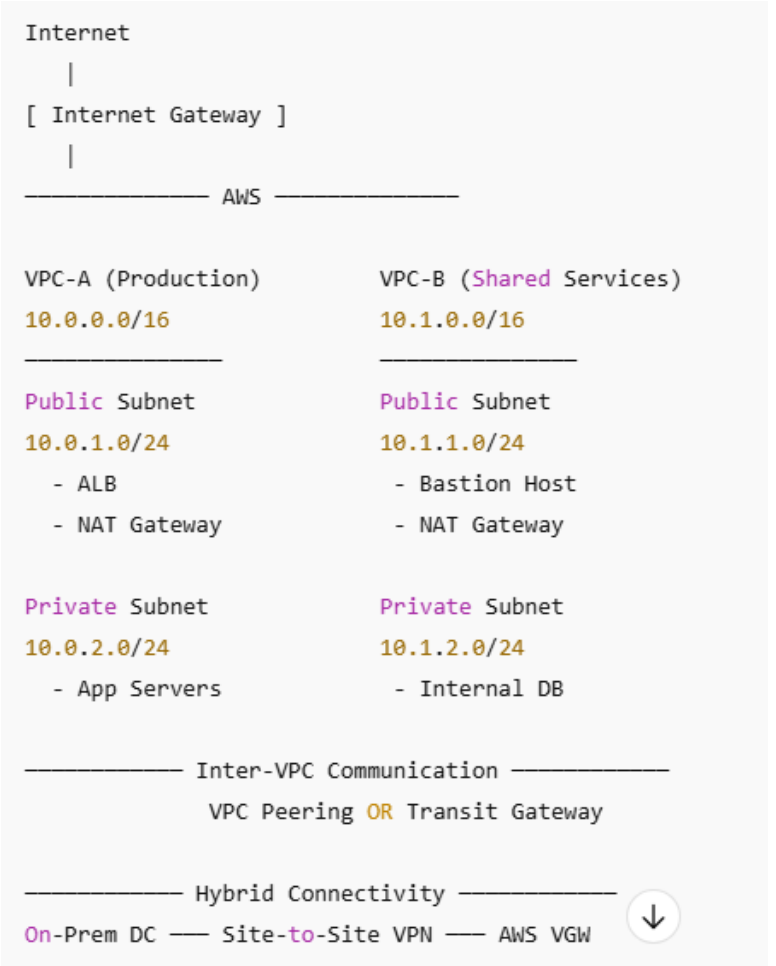
Multi-VPC Hybrid Architecture

Scenario - A company has:

- A Production VPC running a public web application and private backend services
- A Shared Services VPC hosting internal tools and databases
- An On-Prem Data Center that must securely access AWS

The system must:

- Serve users from the internet
- Keep backend systems private
- Allow VPC-to-VPC communication
- Allow On-Prem ↔ VPC communication
- Allow private resources to access the internet securely.



Multi-VPC Hybrid Architecture

VPC-A - Production VPC (10.0.0.0/16)

- Public Subnet (10.0.1.0/24)
- Purpose: Internet-facing resources
- Contains:
  - Application Load Balancer (ALB)
  - NAT Gateway

Route Table:

10.0.0.0/16 → local

0.0.0.0/0 → Internet Gateway

A subnet is public because of its route table, not because of its CIDR.

Multi-VPC Hybrid Architecture

VPC-A - Private Subnet (10.0.2.0/24)

Purpose: Backend services

Contains: EC2 App Servers

No public Ips

Route Table:

10.0.0.0/16 → local

0.0.0.0/0 → NAT Gateway

10.1.0.0/16 → Transit Gateway

Traffic behavior:

Outbound internet → NAT Gateway

Inter-VPC traffic → TGW

No inbound internet traffic

Multi-VPC Hybrid Architecture

VPC-B: Shared Services VPC (10.1.0.0/16)

Public Subnet (10.1.1.0/24) Contains:

Bastion Host (SSH entry point)

NAT Gateway

Route Table:

10.1.0.0/16 → local

0.0.0.0/0 → Internet Gateway

Bastion hosts should live in public subnets, but target private resources.

Multi-VPC Hybrid Architecture

VPC-B: Shared Services VPC (10.1.0.0/16)

Private Subnet (10.1.2.0/24) Contains:

Internal databases

Monitoring tools

Route Table:

10.1.0.0/16 → local

0.0.0.0/0 → NAT Gateway

10.0.0.0/16 → Transit Gateway

On-Prem ↔ AWS Communication (Hybrid) - Site-to-Site VPN

Customer Gateway (On-Prem)

Virtual Private Gateway (AWS)

IPSec tunnel - Traffic flow: On-Prem → VPN Tunnel → VGW → VPC-A / VPC-B